DIGITAL NOTES • SOFT COMPUTING

UNIT – I

Introduction to Soft Computing

What is inside this unit

1	Introduction – What is soft computing?
2	“Soft” vs “Hard” computing
3	Evolutionary computing
4	Soft computing methods (FL, ANN, GA, Swarm, Rough sets)
5	Characteristics of soft computing
6	Recent trends in soft computing
7	Applications of soft computing techniques
8	Revision sheet & exam questions

After this unit you will be able to

- Define soft computing and distinguish it from hard computing.
- Explain evolutionary computing and its main paradigms.
- Name the soft computing methods with their strengths and weaknesses.
- List characteristics, recent trends and real applications.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Introduction to Soft Computing

Many real problems (driving a car, recognising a face, predicting power demand) have **noisy data, vague words and no exact formula**. Classical computing struggles here. **Soft Computing (SC)** is a family of techniques that **tolerate imprecision, uncertainty and partial truth** to give practical, low-cost, robust solutions.

- **Coined by:** Prof. Lotfi A. Zadeh (1992, Berkeley), the father of fuzzy logic.
- **Guiding principle:** *"Exploit the tolerance for imprecision, uncertainty and partial truth to achieve tractability, robustness and low solution cost."*
- **Inspiration:** the human mind and nature (brain → neural nets, evolution → GA, bird flocks → PSO, human language → fuzzy logic).
- **Goal:** "good-enough" answers quickly instead of perfect answers that take forever.

1.1 Why do we need soft computing?

- Real systems are **too complex** to model with exact equations (weather, economy, human body).
- Sensor data is **noisy or incomplete**; human knowledge is expressed in **words**, not numbers.
- Exact methods become impossibly slow for large problems (e.g. travelling salesman is **NP-hard**).
- We often only need a **good, fast, cheap** answer – not the mathematically perfect one.

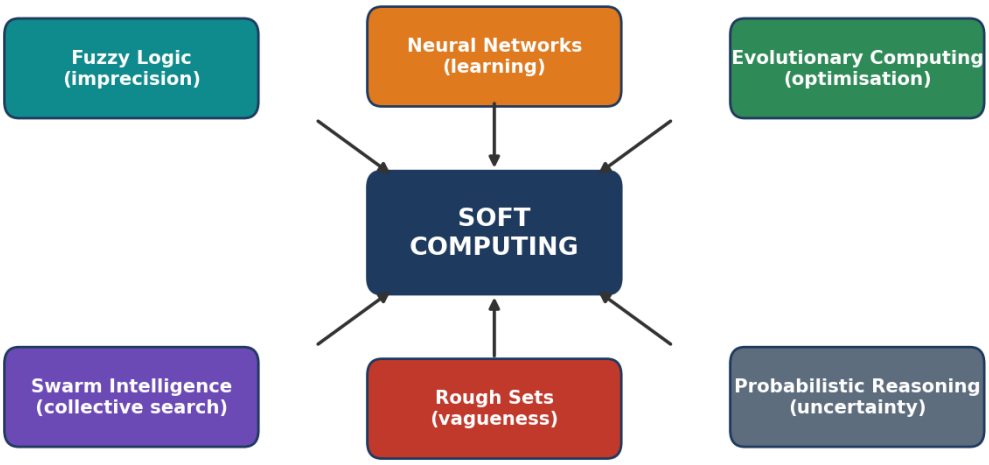


*Fig 1.1 Constituents (partners) of Soft Computing – they work **together**, not in competition*

Exam tip

Say: **SC = Fuzzy Logic + Neural Networks + Evolutionary Computing (+ Probabilistic reasoning, Swarm, Rough sets)**. Zadeh's point: these are *complementary*, so the best systems are **hybrids**.

1.2 Fuzzy idea in one picture

Crisp logic says a person is either tall (1) or not tall (0). Human thinking is gradual – 168 cm is “somewhat tall”. Fuzzy logic captures this with a degree between 0 and 1.

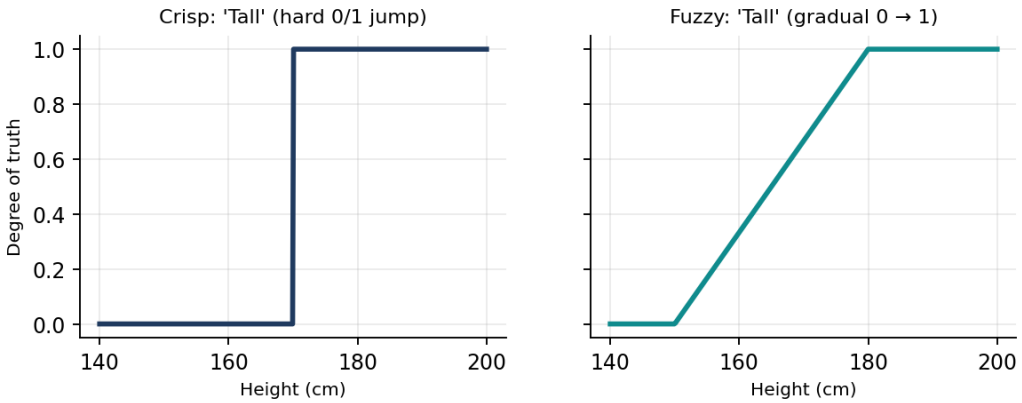


Fig 1.2 Crisp set vs fuzzy set for the word “Tall”

1.3 Everyday example – fuzzy washing machine

The machine does not know exact grams of dirt. It only senses “how dirty” and “how much load” and uses rules written in words. This is soft computing in your home.

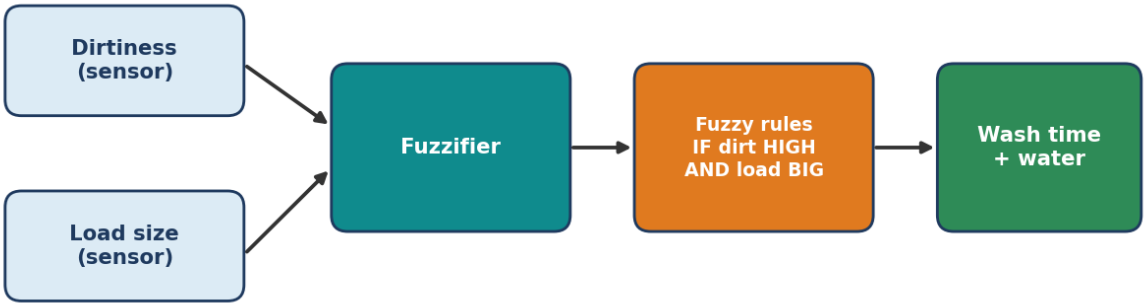


Fig 1.3 Block diagram of a fuzzy washing machine controller

2 “Soft” Computing versus “Hard” Computing

Hard computing is conventional computing: precise models, exact algorithms, Boolean (true/false) logic. **Soft computing** accepts approximation and learns from data/experience.

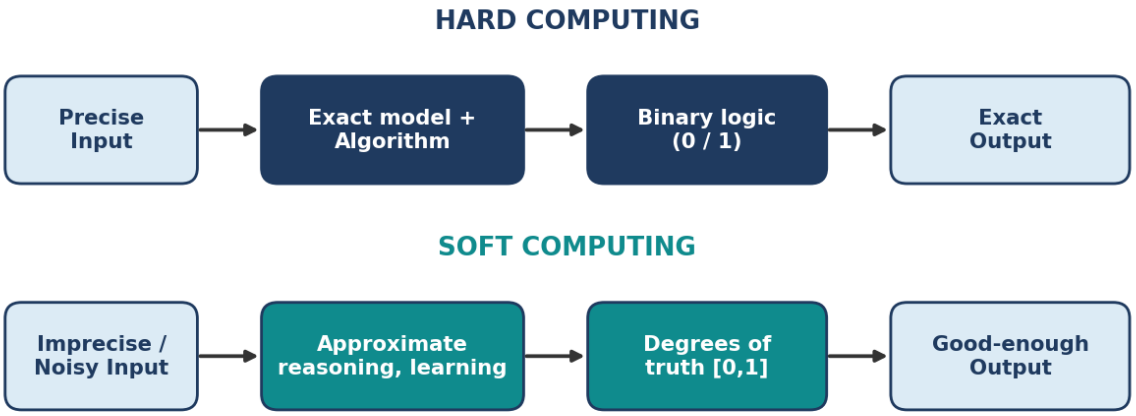


Fig 2.1 Processing style: hard computing vs soft computing

Point	Hard Computing	Soft Computing
Basis	Precision, certainty, rigour	Tolerance to imprecision, uncertainty
Logic	Two-valued (Boolean) – 0 or 1	Multi-valued – degree of truth in [0, 1]
Model	Needs exact mathematical model	Model-free / learns the model from data
Input data	Must be exact and complete	Can be noisy, incomplete, vague
Computation	Sequential, deterministic	Parallel, stochastic (random element)
Output	Exact, same every run	Approximate, may vary slightly
Adaptability	Fixed program, no learning	Adapts and learns (neural nets, GA)
Cost / time	High for complex real problems	Low – quick “good enough” answer
Examples	Payroll, sorting, numerical analysis	Face recognition, fuzzy AC, stock prediction

Memory trick

Hard = “Exact, Fixed, Slow”. Soft = “Flexible, Learns, Fast-enough”. Soft computing is not “weak” computing – it is smarter handling of messy real-world data.

3 Evolutionary Computing

Evolutionary Computing (EC) solves problems by copying **Darwin’s “survival of the fittest”**. A population of candidate solutions evolves over generations: good solutions reproduce, weak ones die out.

3.1 Biological idea → computing idea

Nature	Computing
Individual / organism	One candidate solution (chromosome)
Population	Set of candidate solutions
Fitness (ability to survive)	Objective value – how good the solution is
Natural selection	Better solutions are chosen as parents
Reproduction (crossover)	Two parents are mixed to make children
Mutation	Small random change to keep diversity

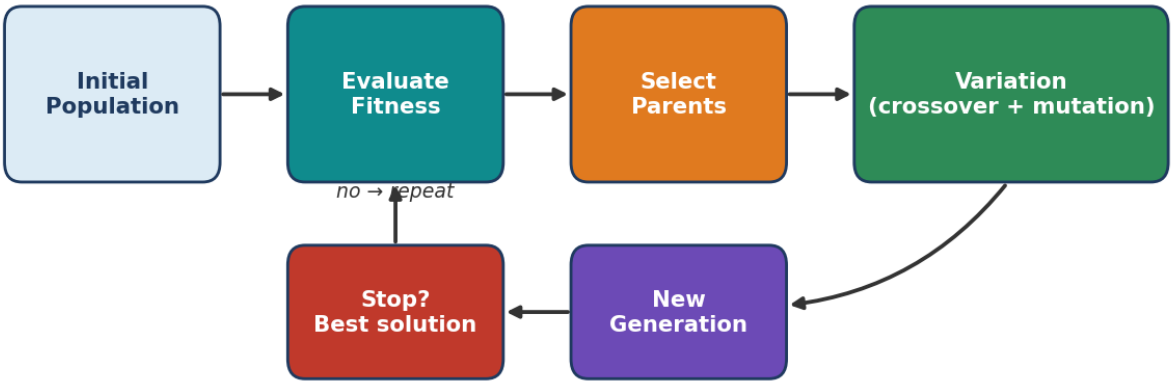


Fig 3.1 Generic evolutionary computing cycle

3.2 Algorithm (pseudo-code)

```
1. t = 0; create random initial population P(0)
2. evaluate fitness of every individual in P(t)
3. WHILE (stopping condition not met) DO
4.    select parents from P(t) based on fitness
5.    apply crossover and mutation → offspring
6.    evaluate offspring; form P(t+1) (best survive)
7.    t = t + 1
8. END WHILE; return the best individual found
```

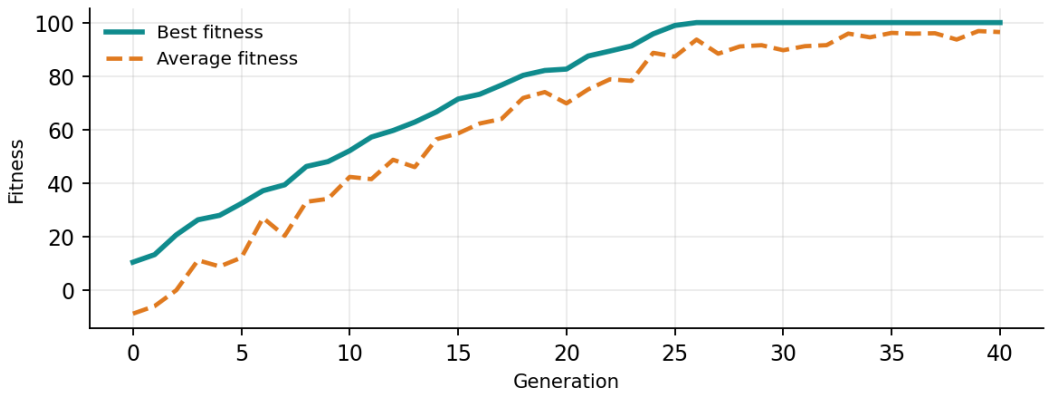


Fig 3.2 Typical behaviour: best fitness rises quickly, then flattens (convergence)

3.3 Main paradigms of EC

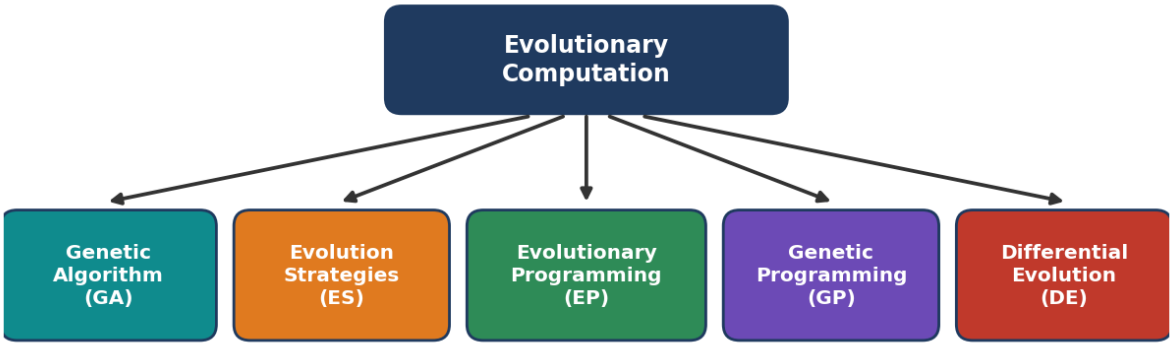


Fig 3.3 Family of evolutionary computation techniques

- **Genetic Algorithm (GA)** – binary/real strings, crossover + mutation (Holland, 1975). Studied fully in Unit IV.
- **Evolution Strategies (ES)** – real-valued vectors, mutation is the main operator, self-adapting step size (Rechenberg).
- **Evolutionary Programming (EP)** – evolves behaviour/finite-state machines, mutation only (Fogel).
- **Genetic Programming (GP)** – evolves *programs* as trees (Koza).
- **Differential Evolution (DE)** – new vector = base + F × (difference of two others); simple and powerful for real-parameter problems.

3.4 Why use EC?

- Works without derivatives – good for non-smooth, discontinuous functions.
- Searches many points at once (population) → avoids getting stuck in one local optimum.
- Handles multi-objective and constrained problems; easy to parallelise.
- **Limitation:** no guarantee of the global optimum; needs many fitness evaluations.

4 Soft Computing Methods

4.1 Fuzzy Logic (FL)

- Handles **vagueness** using membership degrees and IF–THEN rules written in words.
- Example: *IF temperature is HIGH THEN fan speed is FAST.*
- Strength: easy to put human expert knowledge into a machine. Weakness: cannot learn by itself. → Unit II & III.

4.2 Artificial Neural Networks (ANN)

- Network of simple neurons inspired by the brain; knowledge is stored in **connection weights**.
- Learns from examples (supervised, unsupervised, reinforcement); good for pattern recognition and prediction.

- Weakness: works as a “black box” – hard to explain why it gave an answer.

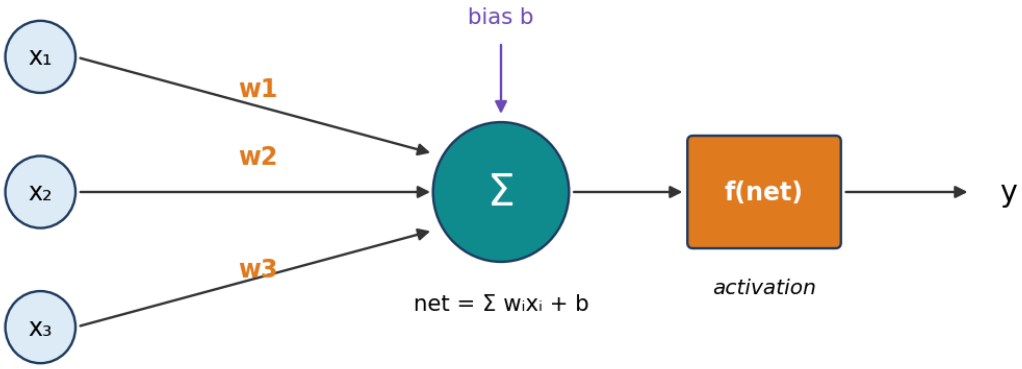


Fig 4.1 Artificial neuron (McCulloch–Pitts / perceptron model): output y = f(Σ wᵢxᵢ + b)

Learning type	Idea	Example
Supervised	Learns from input + correct output (teacher)	Spam detection, perceptron, backpropagation
Unsupervised	Finds hidden groups without labels	K-means, Self-Organising Map
Reinforcement	Learns by reward and punishment	Game playing, robot walking

Activation functions decide how strongly a neuron “fires”. Step gives 0/1; sigmoid squashes to (0,1); tanh to (−1,1); ReLU passes only positive values.

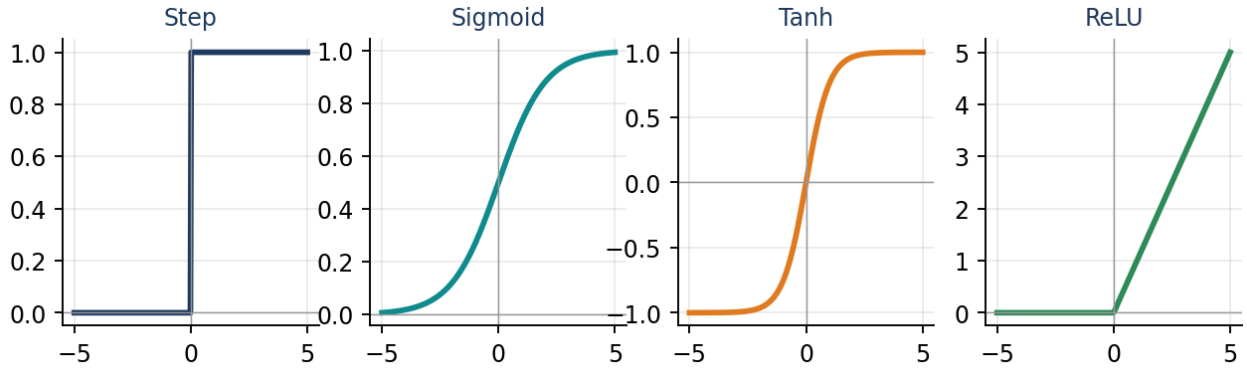


Fig 4.1b Common activation functions

4.3 Evolutionary / Genetic Computing

- Population-based random search for **optimisation** (Section 3).
- Used to tune fuzzy rules, neural weights, schedules, designs.

4.4 Swarm Intelligence

- Simple agents following local rules create **smart group behaviour**.
- **PSO** (bird flocking) and **ACO** (ants finding shortest path). PSO is covered in Unit III.

4.5 Rough Sets and Probabilistic Reasoning

- **Rough sets** (Pawlak, 1982) – describe a concept using lower/upper approximations; no extra data needed (Unit V).
- **Probabilistic reasoning** – Bayesian networks, belief networks; handle uncertainty about *events* (fuzzy handles vagueness of *words*).

Fuzzy logic	Probability
Measures **degree of truth** (how much?)	Measures **likelihood** of an event (how likely?)
"Tea is 0.7 hot" – the tea is already there	"70% chance it will rain" – event not yet happened
Deals with vagueness of language	Deals with randomness / chance

Method	Handles	Strength	Weakness
Fuzzy Logic	Vagueness	Human-like rules, interpretable	No learning
Neural Network	Noisy data	Learns, generalises	Black box
Genetic Algorithm	Optimisation	Global search	Slow, no guarantee
Swarm (PSO/ACO)	Optimisation	Few parameters, fast	Premature convergence
Rough Sets	Incomplete data	Finds reducts & rules	Needs discrete data

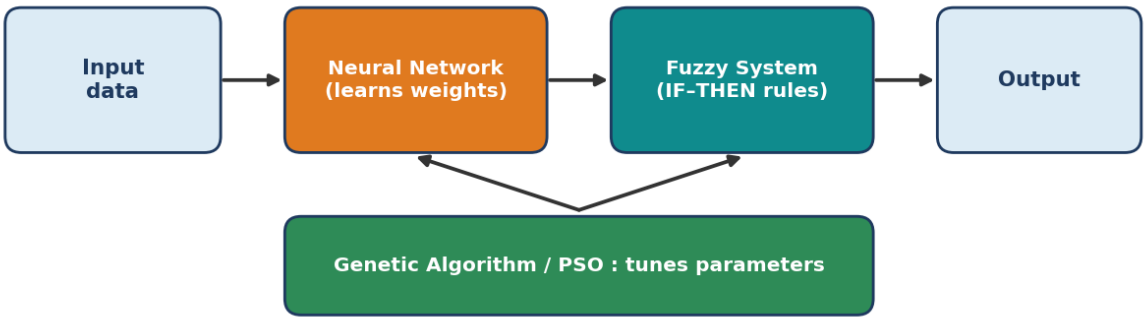


Fig 4.2 A hybrid soft-computing system: neuro-fuzzy model whose parameters are tuned by GA/PSO

Key idea – Hybridisation

Each tool covers another's weakness: **Fuzzy + NN = Neuro-Fuzzy** (learns and explains), **GA + Fuzzy = Genetic-Fuzzy** (auto-tunes rules), **GA + NN = Neuro-Genetic** (finds best weights/architecture).

5 Characteristics of Soft Computing

- **Tolerance to imprecision and uncertainty** – works with approximate, noisy data.
- **Human-expertise based** – uses linguistic knowledge such as "high", "slightly low".
- **Biologically inspired** – brain, evolution, swarms, immune system.

- **Learning and adaptation** – improves with experience; adapts to a changing environment.
- **Model-free** – no exact mathematical model of the system is required.
- **Robustness and fault tolerance** – small errors or missing data do not crash the system.
- **Parallelism** – many computations run together (neurons, population members).
- **Non-linear and stochastic** – can deal with complex non-linear problems using randomness.
- **Low solution cost** – cheap, quick, "good-enough" solutions.
- **Hybrid nature** – techniques combine to form powerful intelligent systems.

5.1 Advantages and limitations

Advantages	Limitations
Handles real-world, noisy, incomplete data	No guarantee of the exact / optimal answer
No exact mathematical model needed	Results can vary between runs (random element)
Cheap, fast, easy to implement	Training / tuning needs time and good data
Learns and adapts to change	Neural nets are hard to explain (black box)
Works well when combined (hybrids)	Choice of parameters is often by trial and error

Quick recall – "TLR-HP"

Tolerance · **L**earning · **R**obustness · **H**uman-like reasoning · **P**arallelism

6 Recent Trends in Soft Computing



Fig 6.1 Milestones in the growth of soft computing

Trend	What it means
Deep learning	Many-layer neural networks (CNN, RNN, Transformers) for images, speech, language
Neuro-fuzzy & ANFIS	Learning ability of NN + readable rules of fuzzy logic
Bio-inspired optimisers	Grey Wolf, Whale, Firefly, Cuckoo Search, Bat algorithm, ABC
Explainable AI (XAI)	Fuzzy rules and rough-set rules used to *explain* black-box decisions
Evolutionary deep learning	AutoML / Neural Architecture Search using GA to design networks
Edge / IoT intelligence	Light-weight fuzzy and neural models running on sensors and phones
Type-2 & intuitionistic fuzzy	Handle uncertainty in the membership function itself
Quantum-inspired & Big-data SC	Quantum-inspired GA/PSO; scalable SC for massive data
Federated & privacy-aware learning	Train models without sharing raw data

7 Applications of Soft Computing Techniques

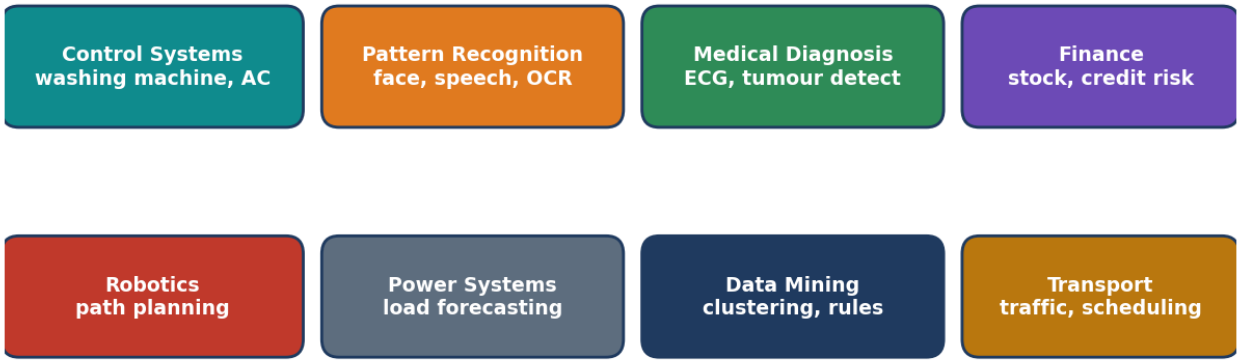


Fig 7.1 Major application areas

Area	Technique	Example
Consumer products	Fuzzy logic	Washing machine sets wash time from dirt and load; rice cooker, AC, camera auto-focus
Industrial control	Fuzzy / Neuro-fuzzy	Cement kiln, steel plant, elevator group control, metro train braking (Sendai)
Medical	ANN, Fuzzy, Rough sets	Cancer detection from scans, ECG classification, drug dosage
Pattern recognition	ANN, Deep learning	Face, fingerprint, handwriting, speech recognition
Power systems	ANN, GA, PSO	Load forecasting, economic load dispatch, fault detection
Finance & business	ANN, GA, Fuzzy	Stock prediction, credit scoring, fraud detection
Robotics & vehicles	Fuzzy, GA, PSO	Path planning, obstacle avoidance, self-driving cars, drones
Scheduling / design	GA, PSO, ACO	Timetabling, travelling salesman, antenna and truss design
Data mining / web	Rough sets, Fuzzy clustering	Rule discovery, recommendation, text classification

7.1 Case study – Sendai subway (Japan)

The Sendai metro (1987) was one of the first big fuzzy-logic successes. A fuzzy controller decides acceleration and braking using rules such as *IF speed is HIGH and distance to station is SHORT THEN brake is STRONG*. Compared with a human driver, it gave **smoother rides, more accurate stops and about 10% less energy use**. The same idea now runs in lifts, cranes and cars (anti-lock braking, automatic gear).

7.2 Case study – Neuro-genetic stock / load prediction

- A neural network predicts next-day electricity demand from past demand, temperature and holidays.
- A genetic algorithm chooses the best network size and learning rate automatically.
- Result: lower forecasting error than fixed manual design – a typical **hybrid** success story.

8 One-page Revision & Important Questions

Revision sheet

- SC: tolerant of imprecision/uncertainty → tractable, robust, low-cost (Zadeh, 1992).
- Hard: exact, Boolean, model needed. Soft: approximate, degrees of truth, learns.
- EC cycle: Initialise → Evaluate → Select → Crossover/Mutate → New generation → Stop?
- ANN = learns; FL = reasons with words; GA/PSO = optimise; Rough sets = approximate concepts.
- Hybrid systems (neuro-fuzzy, genetic-fuzzy, neuro-genetic) give the best of each.
- Trends: deep learning, XAI, bio-inspired optimisers, edge/IoT, type-2 fuzzy.

Quick glossary

Term	Meaning
Imprecision	Not exact (“about 30 °C”)
Uncertainty	Not sure whether something is true
Partial truth	Truth value between completely true and completely false
Tractability	Solvable in reasonable time
Robustness	Keeps working despite noise or small faults
Fitness	Score telling how good a solution is
Hybrid system	Combination of two or more SC techniques

Quick self-check (answers are in the notes)

Q1. Who coined the term soft computing and when? → **L. A. Zadeh, 1992**
Q2. Which logic does hard computing use? → **Two-valued (Boolean)**
Q3. Name the main operators of evolutionary algorithms. → **Selection, crossover, mutation**
Q4. Which method is called a “black box”? → **Neural networks**
Q5. Why hybridise? → **Each technique covers the weakness of the others**

Likely exam questions

- Define soft computing. How does it differ from hard computing? (5–10 marks – use the table + Fig 2.1)
- Explain the characteristics of soft computing.
- What is evolutionary computing? Explain its cycle with a neat diagram.
- List and briefly explain the constituents/methods of soft computing.
- Discuss recent trends and applications of soft computing.
- Why are hybrid systems needed? Give three examples.

DIGITAL NOTES • SOFT COMPUTING

UNIT – II

Fuzzy Systems

What is inside this unit

1	Fuzzy sets – membership, terms, operations, properties
2	Fuzzy relations – composition, properties
3	Fuzzy logic – linguistic variables, hedges, reasoning
4	Fuzzy rule-based systems – architecture, Mamdani vs Sugeno, defuzzification
5	Solved numerical problems
6	Revision sheet & exam questions

After this unit you will be able to

- Represent concepts with fuzzy sets and apply union, intersection, complement.
- Compute max–min composition of fuzzy relations.
- Use linguistic variables, hedges and approximate reasoning.
- Explain the architecture of a fuzzy system and solve a Mamdani example with defuzzification.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Fuzzy Sets

A **crisp (classical) set** has sharp boundaries: an element is either **in (1)** or **out (0)**. A **fuzzy set** (Zadeh, 1965) allows **partial membership** – each element has a **membership degree μ between 0 and 1**.

Crisp set	Fuzzy set
μ(x) ∈ {0, 1}	μ(x) ∈ [0, 1]
Sharp boundary	Gradual / soft boundary
"Age ≥ 18 is adult"	"Young", "Old", "Tall" – no exact cut-off
Obeys excluded-middle: A ∪ A′ = X	A ∪ A′ ≠ X in general

1.1 Notation

- Discrete universe: **A = { μ(x₁)/x₁ + μ(x₂)/x₂ + … }** – the "/" and "+" are only symbols, not division/addition.
- Example: **Young = {1/10 + 0.8/20 + 0.5/30 + 0.1/40}** means a 20-year-old is "young" to degree 0.8.
- Continuous universe: A = ∫ μ(x)/x.

1.2 Important terms of a fuzzy set

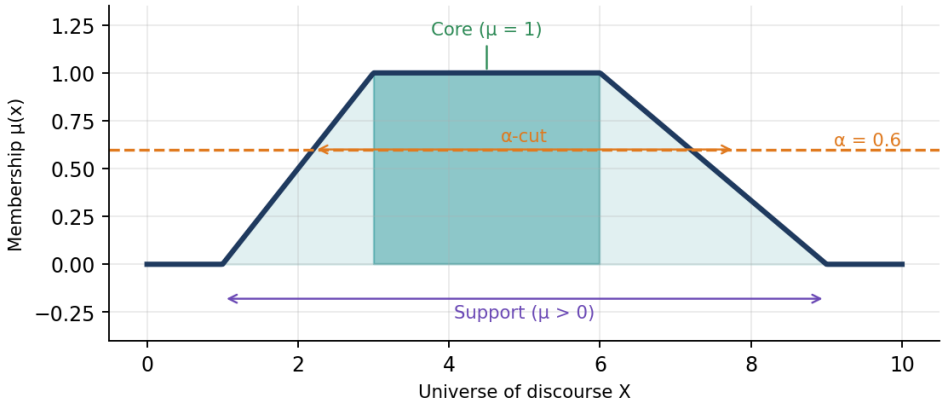


Fig 1.1 Support, core and α-cut of a fuzzy set

Term	Definition	Value in Fig 1.1
Support	All x with μ(x) > 0	[1, 9]
Core	All x with μ(x) = 1	[3, 6]
Height	Maximum membership value	1 (so the set is **normal**)
α-cut (A_α)	Crisp set of x with μ(x) ≥ α	α = 0.6 → [2.2, 7.8]
Crossover point	x where μ(x) = 0.5	—
Cardinality	|A| = Σ μ(x) (scalar)	—

1.3 Membership functions (MF)

The **membership function** maps every element to its degree of membership. Choose the shape by the application; triangular and trapezoidal are the most popular because they are **simple and fast**.

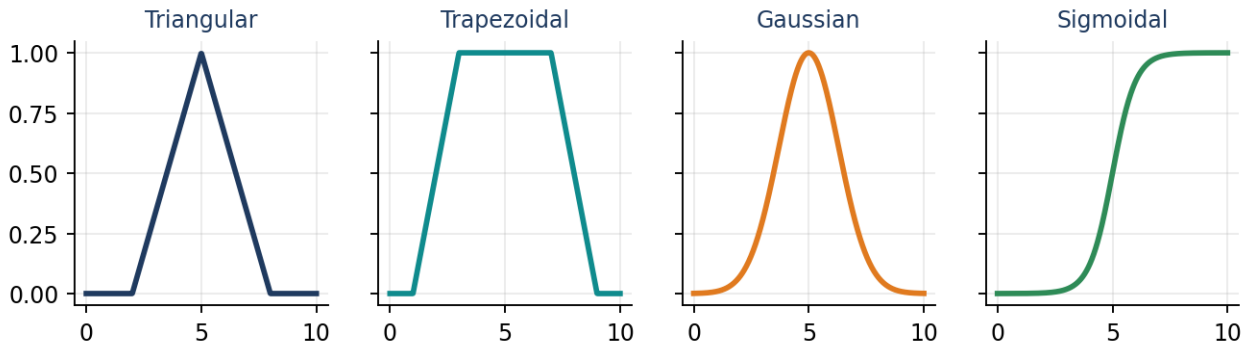


Fig 1.2 Common membership function shapes

- **Triangular** (a, b, c): rises from a to peak b, falls to c.
- **Trapezoidal** (a, b, c, d): flat top between b and c.
- **Gaussian**: smooth bell, $\mu = \exp(-(x-c)^2/2\sigma^2)$ – good for smooth control.
- **Sigmoidal**: S-shaped, used for "open-ended" terms like *very high*.

1.4 Operations on fuzzy sets

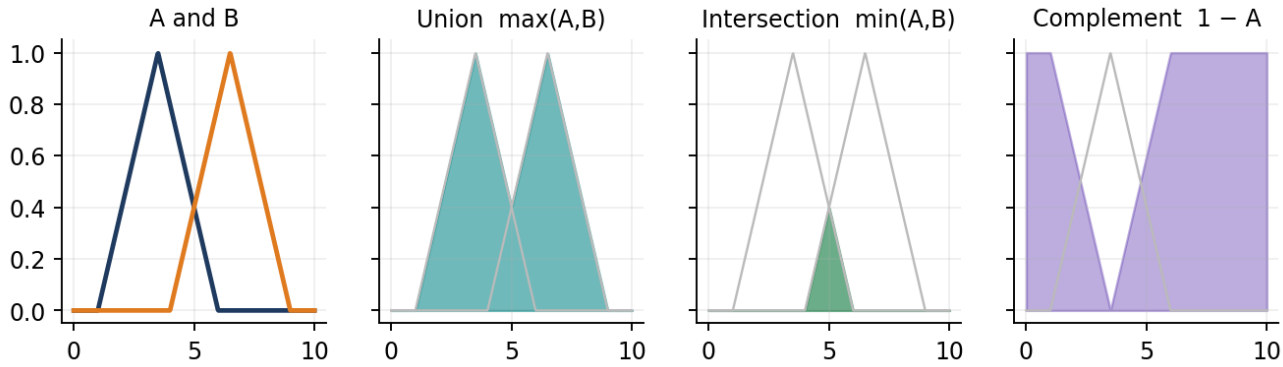


Fig 1.3 Standard fuzzy operations (Zadeh)

Operation	Formula (for every x)	Meaning
Union A ∪ B	$\mu = \max[\ \mu_A(x), \mu_B(x)\ ]$	OR
Intersection A ∩ B	$\mu = \min[\ \mu_A(x), \mu_B(x)\ ]$	AND
Complement A′	$\mu = 1 - \mu_A(x)$	NOT
Algebraic product	$\mu_A(x) \cdot \mu_B(x)$	soft AND
Algebraic sum	$\mu_A + \mu_B - \mu_A\mu_B$	soft OR
Bounded sum	$\min[1, \mu_A + \mu_B]$	—
Bounded difference	$\max[0, \mu_A + \mu_B - 1]$	—

Worked example. A = {0.2/a + 0.7/b + 1/c}, B = {0.5/a + 0.4/b + 0.6/c}

- A ∪ B = {0.5/a + 0.7/b + 1/c}
- A ∩ B = {0.2/a + 0.4/b + 0.6/c}
- A′ = {0.8/a + 0.3/b + 0/c}

Worked example – α-cut and hedges

Tall = {0.1/160 + 0.4/165 + 0.7/170 + 0.9/175 + 1/180}
α-cut (α = 0.5): keep elements with μ ≥ 0.5 → **{170, 175, 180}**
very Tall = μ² → {0.01/160 + 0.16/165 + 0.49/170 + 0.81/175 + 1/180}
somewhat Tall = √μ → {0.32/160 + 0.63/165 + 0.84/170 + 0.95/175 + 1/180}
Notice: "very" makes the set smaller / stricter, "somewhat" makes it larger.

1.5 Properties

Property	Statement
Commutative	A ∪ B = B ∪ A ; A ∩ B = B ∩ A
Associative	(A ∪ B) ∪ C = A ∪ (B ∪ C)
Distributive	A ∪ (B ∩ C) = (A ∪ B) ∩ (A ∪ C)
Idempotent	A ∪ A = A ; A ∩ A = A
De Morgan	(A ∪ B)′ = A′ ∩ B′ ; (A ∩ B)′ = A′ ∪ B′
Involution	(A′)′ = A
Not valid	A ∪ A′ ≠ X and A ∩ A′ ≠ ∅ (excluded-middle & contradiction fail)

Exam tip

The only laws that **fail** for fuzzy sets are **A ∪ A′ = X** and **A ∩ A′ = ∅**. Write this line in every answer on properties.

2 Fuzzy Relations

A **relation** links elements of two sets. A **crisp relation** says "related (1) or not (0)". A **fuzzy relation** R on X × Y gives a **strength of relationship $\mu_R(x, y) \in [0, 1]$**, stored as a **matrix**.

	y_1	y_2	y_3
x_1	0.3	0.7	0.2
x_2	0.8	0.4	0.5

*Fuzzy relation R from $X = \{x_1, x_2\}$ to $Y = \{y_1, y_2, y_3\}$. Example: "x is **close to** y".*

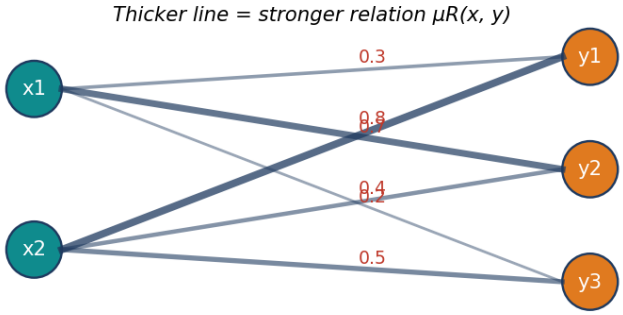


Fig 2.1 Graph view of the fuzzy relation R (weights are membership values)

2.1 Operations on fuzzy relations

- **Union**: R ∪ S = max(R, S) **Intersection**: R ∩ S = min(R, S) **Complement**: R′ = 1 − R
- **Cartesian product** of fuzzy sets A and B: $\mu_{A\times B}(x, y) = \min[\ \mu_A(x), \mu_B(y)\ ]$
- **Inverse** R^{-1}: transpose of matrix R.

2.2 Composition of fuzzy relations

Given R on X × Y and S on Y × Z, the composition **R ∘ S** on X × Z tells how x is related to z **through** y.

- **Max–min:** $\mu_{R\circ S}(x, z) = \max_y \{\ \min[\ \mu_R(x, y), \mu_S(y, z)\ ]\ \}$ (most used)
- **Max–product:** $\mu_{R\circ S}(x, z) = \max_y \{\ \mu_R(x, y) \cdot \mu_S(y, z)\ \}$

Worked example – max–min composition

R (2×3) = [0.30 0.70 0.20] | [0.80 0.40 0.50]
S (3×2) = [0.60 0.90] | [0.20 0.50] | [0.70 0.10]
Step: $(R\circ S)(x_1,z_1) = \max[\ \min(0.3,0.6), \min(0.7,0.2), \min(0.2,0.7)\ ] = \max[0.3, 0.2, 0.2] =$ **0.3**
Repeat for every cell → **R∘S = [0.30 0.50] | [0.60 0.80]** (max–product gives [0.18 0.35] | [0.48 0.72])
*Method: it is like matrix multiplication – replace "×" by **min** and "+" by **max**.*

2.3 Properties of a fuzzy relation (on X × X)

Property	Condition
Reflexive	$\mu_R(x, x) = 1$ for all x
Symmetric	$\mu_R(x, y) = \mu_R(y, x)$
Transitive (max–min)	$\mu_R(x, z) \geq \max_y \min[\ \mu_R(x, y), \mu_R(y, z)\ ]$
Fuzzy equivalence	Reflexive + symmetric + transitive (used for fuzzy clustering)
Fuzzy tolerance	Reflexive + symmetric only

3 Fuzzy Logic

Fuzzy logic is a many-valued logic where a statement can be **partly true**. It lets machines reason with human words like “fairly hot” or “very fast”.

3.1 Linguistic variables

A **linguistic variable** takes **words** as values; each word is a fuzzy set. Example: variable **Temperature** with terms COLD, WARM, HOT.

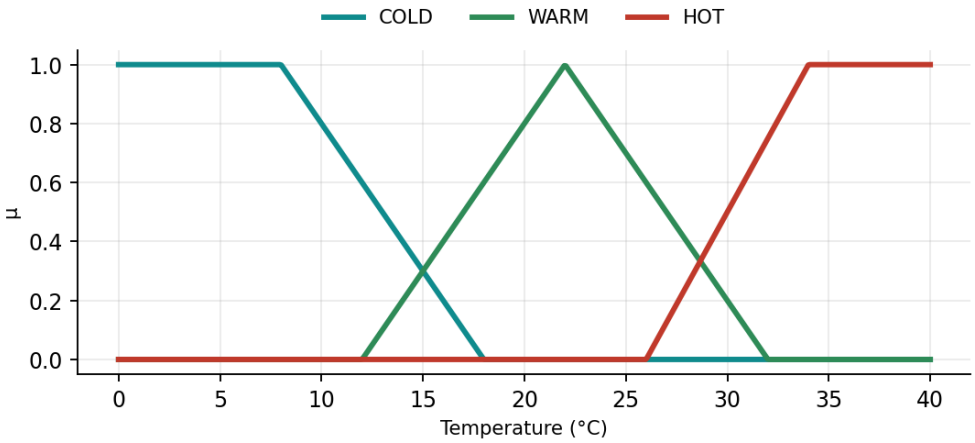


Fig 3.1 Linguistic variable “Temperature” and its three fuzzy terms

3.1b Fuzzy numbers

- A **fuzzy number** is a normal, convex fuzzy set on the real line, e.g. “**about 5**” = triangular (3, 5, 7).
- **Addition** of triangular numbers $(a_1,b_1,c_1) + (a_2,b_2,c_2) = (a_1+a_2, b_1+b_2, c_1+c_2)$. Example: about 5 + about 2 = (3,5,7)+(1,2,3) = **(4, 7, 10)**.
- **Extension principle:** lets any crisp function f(x) be applied to fuzzy sets – $\mu_{f(A)}(y)$ = max over all x with f(x)=y of $\mu_A(x)$.

3.2 Linguistic hedges

Hedge	Operation on μ	Example
very	μ^2 (concentration)	very tall = $(\mu_{tall})^2$
extremely	μ^3	extremely tall
somewhat / more or less	$\sqrt{\mu}$ (dilation)	somewhat tall = $\sqrt{\mu_{tall}}$
not	$1 - \mu$	not tall
slightly	$\sqrt{\mu} \cap$ not(very μ)	slightly tall

3.3 Fuzzy propositions and connectives

- **Atomic:** “x is A” → truth = $\mu_A(x)$. **AND** = min, **OR** = max, **NOT** = $1 - \mu$.

- **Fuzzy implication** "IF p THEN q": **Mamdani** $\min(\mu_p, \mu_q)$; **Larsen** product $\mu_p \cdot \mu_q$; **Zadeh** $\max[\min(\mu_p, \mu_q), 1 - \mu_p]$; **Kleene-Dienes** $\max(1 - \mu_p, \mu_q)$.

3.4 Approximate reasoning – Generalised Modus Ponens (GMP)

	Classical Modus Ponens	Generalised Modus Ponens (fuzzy)
Rule	IF x is A THEN y is B	IF x is A THEN y is B
Fact	x is A	x is **A′** (close to A)
Conclusion	y is B	y is **B′** (close to B)

The conclusion is computed by the **compositional rule of inference**: **B′ = A′ ∘ R**, where R is the fuzzy relation made from the implication A → B.

> **Remember**
> Classical logic needs an exact match of fact and rule. Fuzzy reasoning works even when the fact is only approximately the same – this is why it suits real life.

4 Fuzzy Rule-Based Systems (FRBS)

A **Fuzzy Rule-Based System** (Fuzzy Inference System / Fuzzy Logic Controller) turns **crisp inputs → crisp output** using IF–THEN rules written in words.

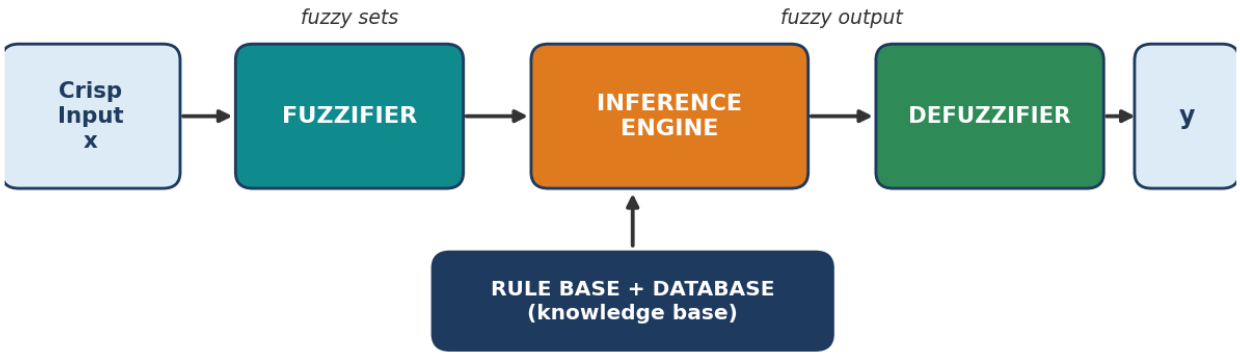


Fig 4.1 Block diagram of a fuzzy rule-based system

Block	Job	Example
Fuzzification	Convert crisp input into membership degrees	28 °C → WARM 0.4, HOT 0.25
Knowledge base	Database (MFs) + rule base (IF–THEN rules from experts)	IF temp is HOT THEN fan is FAST
Inference engine	Fire the rules, combine their results	min for AND, max to combine rules
Defuzzification	Convert fuzzy output into one crisp number	Centroid → one crisp fan speed

4.1 Steps of fuzzy inference

- **Step 1 – Fuzzify** the inputs using membership functions.
- **Step 2 – Apply operators** (AND = min, OR = max) to get each rule's **firing strength**.
- **Step 3 – Implication:** clip (min) or scale (product) each rule's output fuzzy set by its firing strength.
- **Step 4 – Aggregate** all clipped outputs using max.
- **Step 5 – Defuzzify** the aggregated set to a crisp value.

4.2 Mamdani vs Sugeno (TSK) models

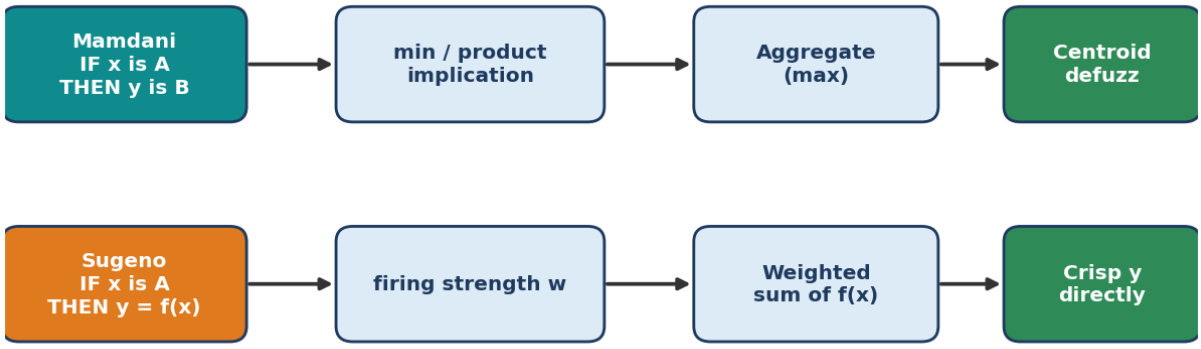


Fig 4.2 Mamdani vs Sugeno inference paths

Point	Mamdani	Sugeno (Takagi–Sugeno–Kang)
Rule consequent	Fuzzy set (IF … THEN y is HIGH)	Function of inputs: $y = a \cdot x_1 + b \cdot x_2 + c$ (or a constant)
Defuzzification	Needed (centroid etc.) – slower	Weighted average $\Sigma w_i y_i / \Sigma w_i$ – fast
Interpretability	High – very human-like	Lower
Best for	Expert-based control, decision support	Adaptive / optimised systems, neuro-fuzzy (ANFIS)

4.3 Worked example – Fan speed controller

Rules: **R1** IF Temp is COLD THEN Fan is SLOW. **R2** IF Temp is WARM THEN Fan is MEDIUM. **R3** IF Temp is HOT THEN Fan is FAST. Input: **28 °C**. Memberships: COLD = 0.00, WARM = 0.40, HOT = 0.25. Each rule clips its output set at its firing strength; the clipped sets are joined with max.

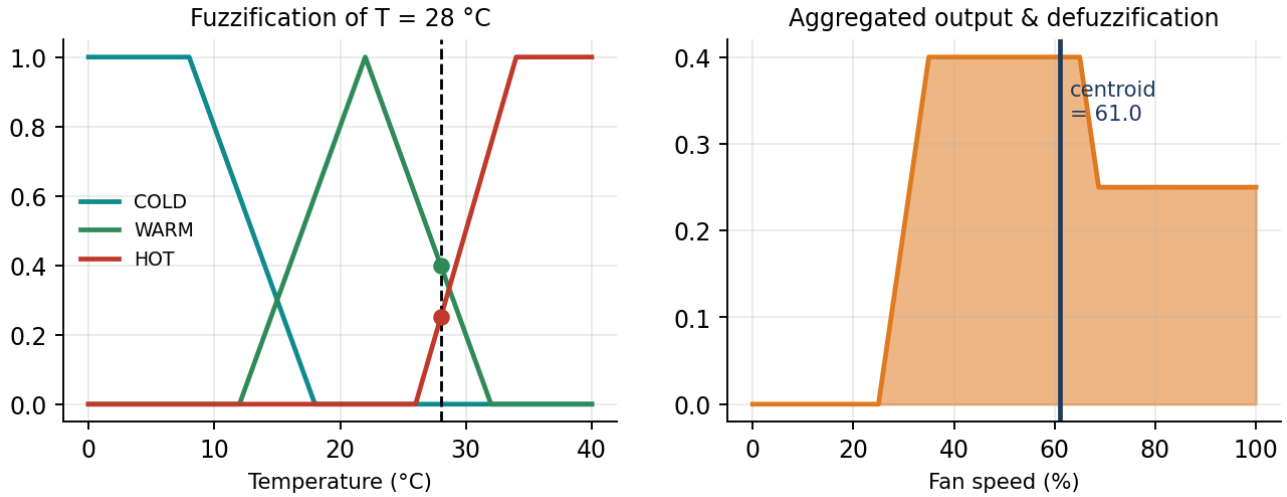


Fig 4.3 Mamdani inference for 28 °C – crisp fan speed ≈ 61 %

4.4 Rule table (FAM) – two-input example

With two inputs the rules are shown as a **Fuzzy Associative Memory (FAM) table**: each cell is one rule, e.g. *IF Service is Good AND Food is Good THEN Tip is MEDIUM*. 3 × 3 terms = 9 rules.

	Food: Poor	Food: Good	Food: Great
Service: Poor	LOW	LOW	MEDIUM
Service: Good	LOW	MEDIUM	HIGH
Service: Great	MEDIUM	HIGH	HIGH

Fig 4.4 FAM table for a tip-advisor fuzzy system

4.5 Defuzzification methods

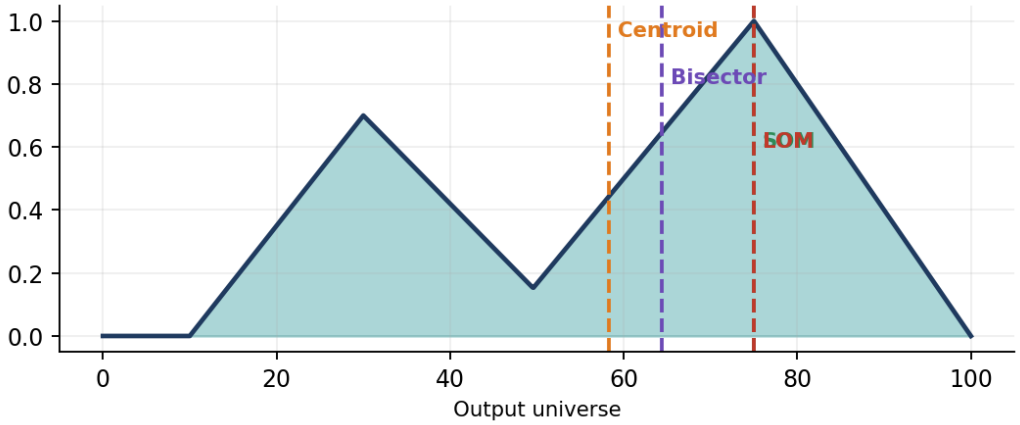


Fig 4.5 Different defuzzifiers give different crisp values for the same fuzzy output

Method	Idea	Formula / note
Centroid (COG, COA)	Centre of gravity of the area – most popular, smooth	$z^* = \int \mu(z) \cdot z \, dz / \int \mu(z) \, dz$
Bisector	Line dividing the area into two equal halves	area left = area right
Mean of Maxima (MOM)	Average of x where μ is maximum	fast, less smooth
Smallest / Largest of Maxima	Left-most / right-most peak point	SOM, LOM
Weighted average	Used with singleton / Sugeno outputs	$z^* = \Sigma \mu_i z_i / \Sigma \mu_i$

Worked example – Sugeno (zero-order) model

R1: IF speed is LOW THEN brake = 10 R2: IF speed is HIGH THEN brake = 80
For one input: firing strengths $w_1 = 0.3$ (LOW), $w_2 = 0.6$ (HIGH)
Output = $(w_1 \cdot 10 + w_2 \cdot 80) / (w_1 + w_2) = (3 + 48) / 0.9 = 56.7$
No clipping, no centroid – just a weighted average, so it is very fast.

4.6 Design steps of a fuzzy controller

- **1.** Identify inputs, outputs and their ranges (universe of discourse).
- **2.** Choose the number and shape of membership functions (3–7 terms is usual).
- **3.** Write the rule base from expert knowledge or data.
- **4.** Select operators: AND (min), implication (min), aggregation (max), defuzzifier (centroid).
- **5.** Simulate, then **tune** MFs and rules (manually or by GA / neural learning).

4.7 Fuzzy controller in a feedback loop

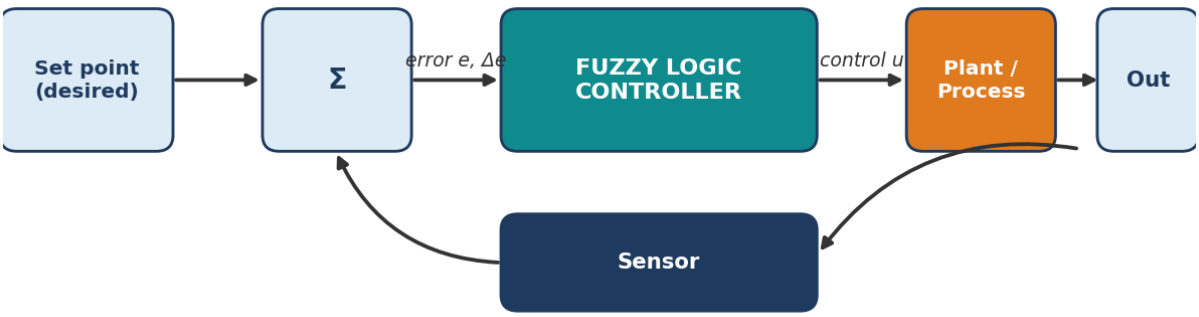


Fig 4.6 Closed-loop control: inputs are usually error (e) and change of error (Δe)

Point	Conventional (PID) controller	Fuzzy logic controller
Needs plant model?	Yes – equations of the process	No – uses expert rules
Non-linear plants	Poor unless re-tuned	Handles well
Noise / sensor errors	Sensitive	Robust
Design effort	Mathematical analysis	Rule writing + tuning
Stability proof	Well established	Difficult

4.8 Advantages, limitations, applications

- **Advantages:** no exact model needed; easy to understand rules; tolerant to noisy sensor data; cheap to implement.
- **Limitations:** rules and MFs are designed by trial/expert; many inputs → **rule explosion** (n inputs × m terms = m^n rules); cannot learn alone.
- **Applications:** washing machine, AC, camera focus, ABS braking, metro control, medical diagnosis, traffic lights.

5 Solved Numerical Problems

Problem 1 – Fuzzy set operations

A = {0.1/1 + 0.6/2 + 1/3 + 0.4/4}, B = {0.5/1 + 0.2/2 + 0.9/3 + 0.7/4}. Find A ∪ B, A ∩ B, A′, and A ∩ B′.
A ∪ B = {0.5/1 + 0.6/2 + 1/3 + 0.7/4} **A ∩ B** = {0.1/1 + 0.2/2 + 0.9/3 + 0.4/4}
A′ = {0.9/1 + 0.4/2 + 0/3 + 0.6/4} **B′** = {0.5/1 + 0.8/2 + 0.1/3 + 0.3/4}
A ∩ B′ = min(A, B′) = {0.1/1 + 0.6/2 + 0.1/3 + 0.3/4}

Problem 2 – Approximate reasoning (compositional rule of inference)

Rule: IF x is A THEN y is B, with A = {0.2/x_1 + 0.8/x_2 + 1/x_3}, B = {0.3/y_1 + 1/y_2}. Fact: x is A′ = {0.1/x_1 + 0.9/x_2 + 0.6/x_3}. Find B′.
Step 1 – relation (Mamdani): R(x,y) = min[A(x), B(y)] → rows x_1,x_2,x_3: [0.2 0.2] | [0.3 0.8] | [0.3 1.0]
Step 2 – compose: B′ = A′ ∘ R (max–min).
B′(y_1) = max[min(0.1,0.2), min(0.9,0.3), min(0.6,0.3)] = **0.3**
B′(y_2) = max[min(0.1,0.2), min(0.9,0.8), min(0.6,1)] = **0.8**
Answer: B′ = {0.3/y_1 + 0.8/y_2}

Problem 3 – Check a relation

R = [[1, 0.8],[0.8, 1]]. Reflexive? diagonal = 1 → **yes**. Symmetric? R_{12} = R_{21} = 0.8 → **yes**. Transitive? R_{12} = 0.8 ≥ max[min(R_{11},R_{12}), min(R_{12},R_{22})] = 0.8 → **yes**. So R is a **fuzzy equivalence relation**.

How to attempt a numerical in the exam

1) Write the formula. 2) Show one full calculation. 3) Give the final answer in the same set notation. Neat steps earn marks even if the final number is wrong.

6 One-page Revision & Important Questions

Formula sheet

• Union = max, Intersection = min, Complement = 1 − μ. • Support: μ>0; Core: μ=1; α-cut: μ≥α.
• Max–min composition: $(R \circ S)(x,z) = \max_y \min[R(x,y), S(y,z)]$. • very = μ^2, somewhat = $\sqrt{\mu}$.
• FRBS: Fuzzify → Fire rules → Aggregate → Defuzzify. • Centroid $z^* = \Sigma\mu(z)\cdot z / \Sigma\mu(z)$.
• Mamdani = fuzzy consequent; Sugeno = linear/constant consequent (weighted average).

Quick self-check

Q1. Which two Boolean laws fail for fuzzy sets? → **Excluded middle and contradiction**
Q2. Which operator replaces AND in fuzzy logic? → **min** (or product)
Q3. Which defuzzifier is most popular? → **Centroid**
Q4. Which model needs no defuzzifier? → **Sugeno**

Likely exam questions

- Define fuzzy set. Differentiate crisp and fuzzy sets. Explain support, core, α-cut with a diagram.
- Explain fuzzy set operations and properties with examples.
- What is a fuzzy relation? Find max–min composition of the given relations (numerical).
- Explain linguistic variables and hedges; state generalised modus ponens.
- Draw and explain the architecture of a fuzzy rule-based system.
- Differentiate Mamdani and Sugeno models; explain defuzzification methods.

DIGITAL NOTES • SOFT COMPUTING

UNIT – III

Fuzzy Decision Making & Particle Swarm Optimization

What is inside this unit

1	Fuzzy decision making – process and types
2	Bellman–Zadeh model (goals & constraints)
3	Individual, multi-person, multi-criteria, multi-stage decisions
4	Particle Swarm Optimization – concept, equations, algorithm
5	PSO example, topologies, PSO vs GA, applications
6	Solved numerical problems
7	Revision sheet & exam questions

After this unit you will be able to

- Model a decision problem with fuzzy goals and constraints (max–min principle).
- Solve simple multi-criteria and group decision problems.
- Explain how PSO works and write its update equations.
- Perform one PSO iteration by hand and compare PSO with GA.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Fuzzy Decision Making

Decision making means choosing the best option from several alternatives. In real life, goals are vague ("cheap", "fast", "good quality") and data is uncertain. **Fuzzy decision making (FDM)** uses fuzzy sets so that goals, constraints and preferences can be expressed in **words and degrees**.



Fig 1.1 General process of fuzzy decision making

Classical decision	Fuzzy decision
Goals and constraints are crisp (cost ≤ 5000)	Goals and constraints are fuzzy (cost *should be about* 5000)
Alternative is either feasible or not	Alternative is feasible to a **degree**
Single sharp optimum	Best **compromise** with highest satisfaction degree

1.1 Types of fuzzy decision making

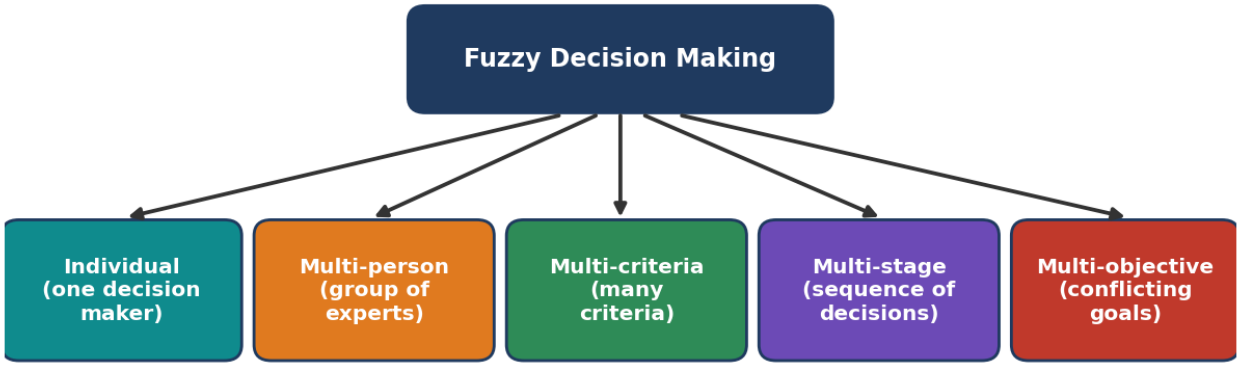


Fig 1.2 Classification of fuzzy decision problems

1.2 Bellman–Zadeh model (fuzzy goals + fuzzy constraints)

Proposed by **Bellman and Zadeh (1970)** – the base of fuzzy decision theory. A **fuzzy decision D** is the **intersection** of fuzzy goal G and fuzzy constraint C:

Key formulas

$\mu_D(x) = \min [\mu_G(x), \mu_C(x)]$ (for several goals/constraints: min over all of them)
Optimal decision: x* = the alternative with **maximum $\mu_D(x)$** ("max–min" principle)

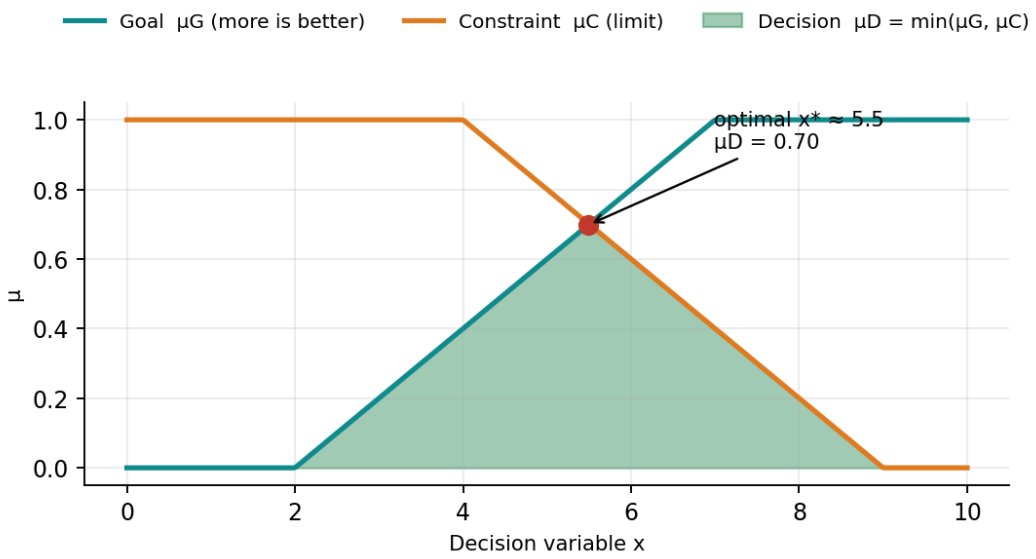


Fig 1.3 Goal and constraint conflict – the best decision is at the peak of their intersection

Worked example – choosing a laptop

Five laptops. μG = how well it meets the goal "high performance", μC = how well it satisfies the constraint "price near budget".
μG = A1: 0.9, A2: 0.7, A3: 0.5, A4: 0.6, A5: 0.3
μC = A1: 0.4, A2: 0.6, A3: 0.8, A4: 0.7, A5: 0.9
μD = min(μG, μC) = A1: 0.4, A2: 0.6, A3: 0.5, A4: 0.6, A5: 0.3
Best choice = A2 (highest μD = 0.6). Note A1 has the best performance but loses because it is too costly.

1.3 Individual decision making

- One decision maker; all alternatives **X = {x₁ … xₙ}** and their fuzzy utility / preference values are known.
- Uses **fuzzy preference relation $R(x_i, x_j)$** = degree to which x_i is preferred over x_j (R = 0.5 → indifferent).
- Ranking: for each alternative find how much it **dominates the others**: $\mu_{nd}(x_i) = 1 - \max_j [R(x_j, x_i) - R(x_i, x_j)]$; the largest value = **non-dominated best**.
- Other ways: ranking fuzzy numbers by **centroid** or **α-cut** values.

Worked example – fuzzy preference relation

Preference of x_i over x_j: R = [[0.5, 0.8, 0.6],[0.2, 0.5, 0.7],[0.4, 0.3, 0.5]] (rows = x₁, x₂, x₃).
Row-wise minimum dominance: x₁ → min(0.5, 0.8, 0.6) = 0.5; x₂ → min(0.2, 0.5, 0.7) = 0.2; x₃ → min(0.4, 0.3, 0.5) = 0.3.
Highest = x₁, so **x₁ is the best alternative**; ranking x₁ > x₃ > x₂.

1.3b Ranking fuzzy numbers

When ratings are triangular fuzzy numbers (a, b, c), convert each to one crisp score and sort. Simple method: **centroid = (a + b + c) / 3**.

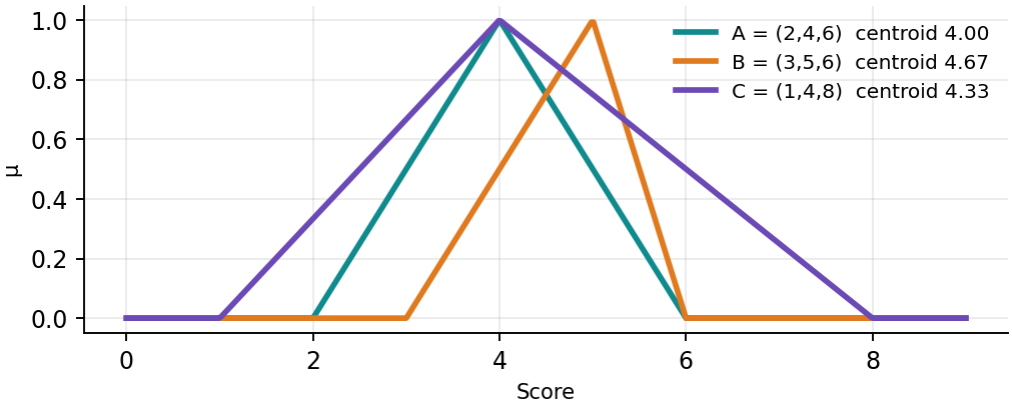


Fig 1.3b Three alternatives rated by triangular fuzzy numbers

Scores: A = 4.00, B = 4.67, C = 4.33 → **Ranking: B > C > A**. B has the highest centroid, so B is chosen.

Linguistic rating	Triangular fuzzy number
Very low	(0, 0, 0.25)
Low	(0, 0.25, 0.5)
Medium	(0.25, 0.5, 0.75)
High	(0.5, 0.75, 1)
Very high	(0.75, 1, 1)

1.4 Multi-person (group) decision making

Several experts give opinions; the system must find a **group consensus**.

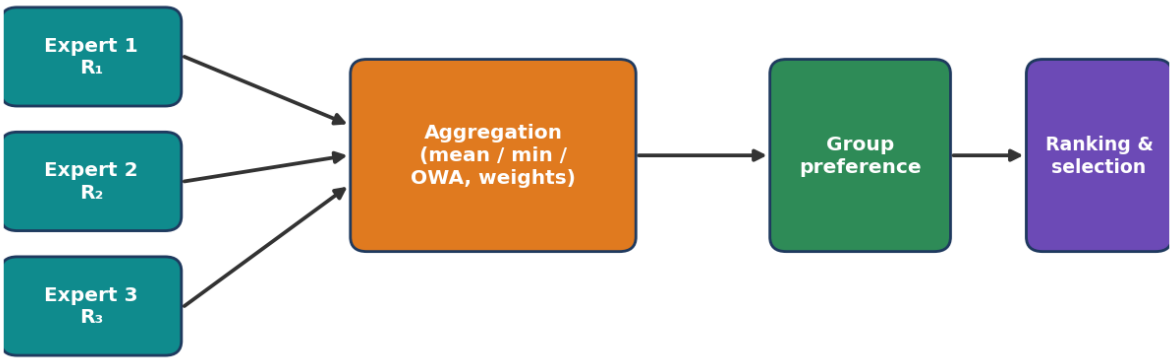


Fig 1.4 Multi-person fuzzy decision making

- Each expert gives a fuzzy preference relation R_k over the alternatives.
- **Aggregate**: arithmetic mean, weighted mean (expert importance), min (cautious) or **OWA** (ordered weighted averaging – “most experts agree”).

- Check **consensus degree**; if low, experts discuss and revise.
- Rank alternatives from the group relation and select the best.

Worked example – aggregating three experts

Experts rate alternatives A, B, C (rows E1, E2, E3): E1 = (0.9, 0.6, 0.3), E2 = (0.7, 0.8, 0.4), E3 = (0.8, 0.5, 0.6)
Mean = (0.80, 0.63, 0.43) → A is best. **Min (all must agree)** = (0.7, 0.5, 0.3) → A still best.
OWA (weights 0.5, 0.3, 0.2 on the sorted scores) for A: 0.9, 0.8, 0.7 → 0.5×0.9 + 0.3×0.8 + 0.2×0.7 = **0.83**

1.5 Multi-criteria decision making (MCDM)

The choice depends on **many criteria** (cost, quality, delivery…) with **different importance (weights)**. Ratings and weights are fuzzy / linguistic.

Step	Action
1	List alternatives and criteria; give each criterion a weight w_j ($\Sigma w = 1$)
2	Rate every alternative on every criterion → membership value μ_{ij} (from words like *good, fair*)
3	Aggregate: **weighted sum** $\Sigma\, w_j\mu_{ij}$ or **weighted min** $\min(\mu_{ij}^{w_j})$
4	Rank by final score; pick the highest

Worked example – supplier selection

Weights: Cost 0.5, Quality 0.3, Delivery 0.2
P = (0.8, 0.6, 0.7) → 0.5×0.8 + 0.3×0.6 + 0.2×0.7 = **0.72**
Q = (0.5, 0.9, 0.8) → 0.25 + 0.27 + 0.16 = **0.68**
R = (0.9, 0.4, 0.5) → 0.45 + 0.12 + 0.10 = **0.67**
Ranking: Supplier P > Supplier Q > Supplier R – Supplier P is selected.

1.5b Fuzzy TOPSIS in brief

- Build the decision matrix of fuzzy ratings and normalise it; multiply by weights.
- Find the **ideal best** (all criteria best) and **ideal worst** solutions.
- Compute distance of each alternative from both: d^+ (to best) and d^- (to worst).
- **Closeness coefficient CC = d^- / (d^+ + d^-)** – the alternative with the largest CC is selected.

1.6 Multi-stage and multi-objective decisions

- **Multi-stage**: decisions in sequence, each affecting the next (fuzzy dynamic programming). Final μ_D = min over all stages.
- **Multi-objective**: conflicting objectives (maximise profit, minimise pollution). Each objective becomes a fuzzy goal; the max–min compromise is chosen.

1.6b Case study – fuzzy loan approval

A bank must decide whether to approve a loan. Strict rules ("income > 30000 AND age < 55") reject good customers who are just outside the limit. A fuzzy system gives **partial credit**.

Criterion	Fuzzy terms	Weight
Income	Low / Medium / High	0.4
Credit history	Poor / Fair / Good	0.35
Existing debt	Light / Heavy	0.25

- Applicant: income = High (0.8), history = Fair (0.6), debt = Light (0.7).
- Score = 0.4×0.8 + 0.35×0.6 + 0.25×0.7 = 0.32 + 0.21 + 0.175 = **0.705**.
- Decision rule: score ≥ 0.65 → **approve**; 0.45–0.65 → refer to manager; below 0.45 → reject. Result: **approve**.

1.7 Applications, advantages and limits

- **Applications:** supplier / project selection, medical diagnosis choice, investment portfolio, site selection, risk assessment, e-commerce recommendation.
- **Advantages:** uses natural language; handles conflicting goals; accepts expert opinion directly.
- **Limitations:** results depend on chosen MFs / weights; many experts or criteria make computation long.

Exam tip

Always write the **max–min principle**: decision = intersection of goals and constraints = **min**; best alternative = **max** of that.

2 Particle Swarm Optimization (PSO)

PSO (Kennedy & Eberhart, **1995**) is a **swarm-intelligence** optimiser inspired by **flocking of birds and schooling of fish**. A swarm of **particles** flies through the search space; each remembers its own best place and also follows the best place found by the group.

- Each particle = one candidate solution, with a **position x** and a **velocity v**.
- **pbest** – best position this particle has visited. **gbest** – best position found by the whole swarm.
- Simple idea: "*I go where I was good before, and where my friends found food*."
- No crossover / mutation, no derivatives; only a few parameters; works for continuous problems.

2.1 Position and velocity update equations

Core equations of PSO

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t))$$
$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Symbol	Meaning	Typical value
w	Inertia weight – keeps old direction; large = explore, small = exploit	0.9 → 0.4 (decreasing)
c_1	Cognitive coefficient – pull towards own best	1.5 – 2.0
c_2	Social coefficient – pull towards swarm best	1.5 – 2.0
r_1, r_2	Random numbers in [0, 1] – give randomness	new every step
Swarm size	Number of particles	20 – 50
v_{max}	Limit on velocity to avoid flying out	10–20% of range

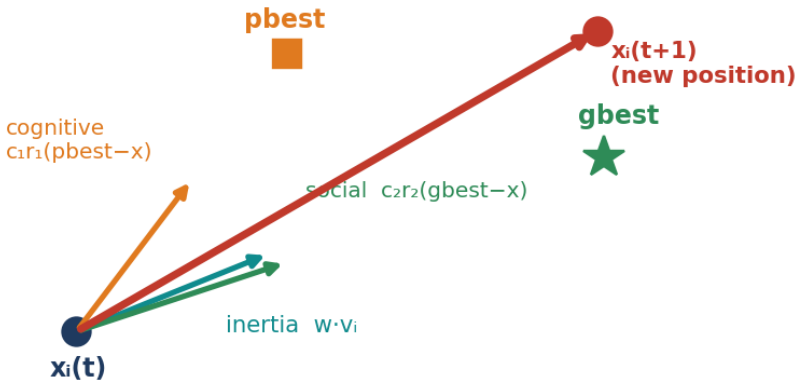


Fig 2.1 New velocity = inertia + cognitive pull + social pull

2.2 PSO algorithm (flowchart)

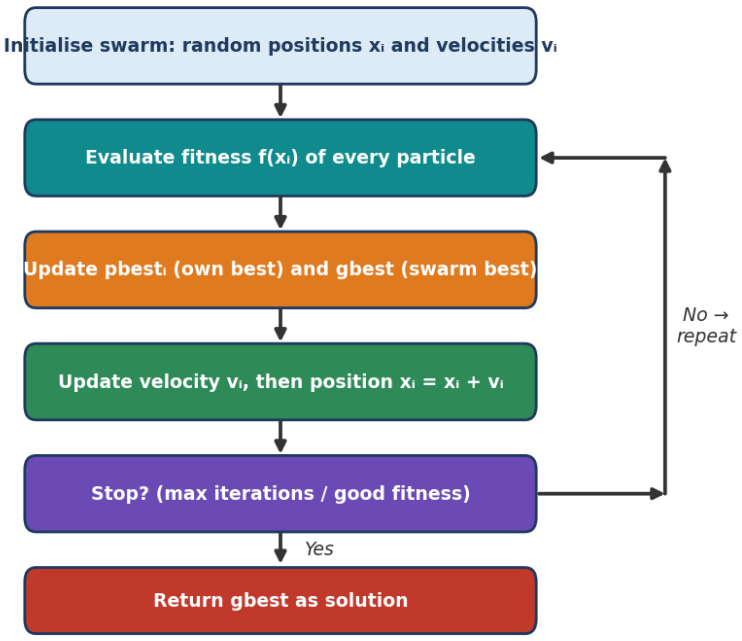


Fig 2.2 Flowchart of PSO

Algorithm PSO
1. for each particle i: random x_i, v_i; $pbest_i = x_i$
2. gbest = best of all pbest
3. repeat until stopping criterion:
   for each particle i:
     update v_i and x_i using the equations
     if $f(x_i)$ better than $f(pbest_i)$: $pbest_i = x_i$
     if $f(pbest_i)$ better than f(gbest): $gbest = pbest_i$
4. output gbest

2.3 Worked numerical example (one particle, one step)

Given

x = 4.0, v = 1.0, pbest = 3.0, gbest = 2.0; w = 0.7, $c_1 = c_2 = 1.5$; $r_1 = 0.4$, $r_2 = 0.6$
v(t+1) = 0.7×1.0 + 1.5×0.4×(3.0−4.0) + 1.5×0.6×(2.0−4.0) = 0.70 + (-0.60) + (-1.80) = **-1.70**
x(t+1) = 4.0 + (-1.70) = **2.30**
The particle moves closer to both pbest (3) and gbest (2).

2.4 Swarm behaviour in action

The figure shows 20 particles minimising $f(x, y) = x^2 + y^2$ (minimum at the ★). They start scattered and **converge** to the optimum within a few tens of iterations.

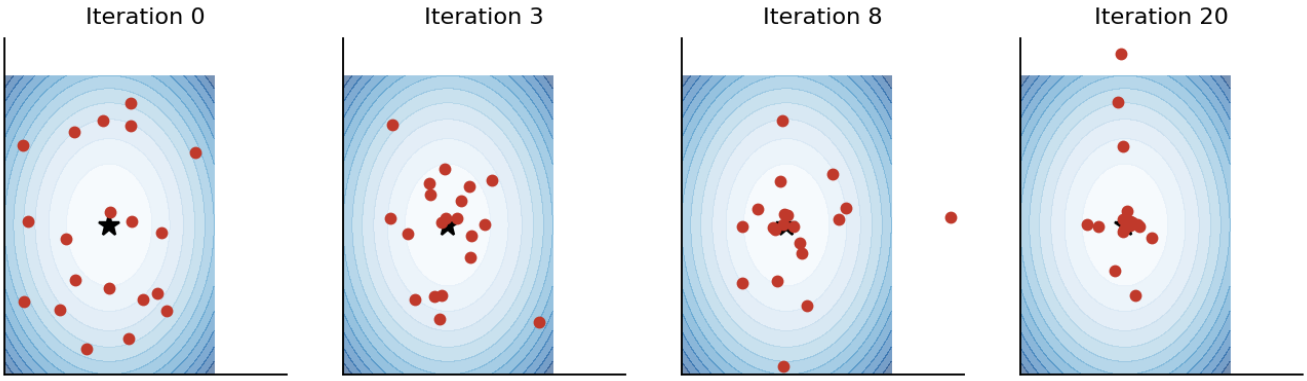


Fig 2.3 Swarm positions at iterations 0, 3, 8 and 20 (simulation)

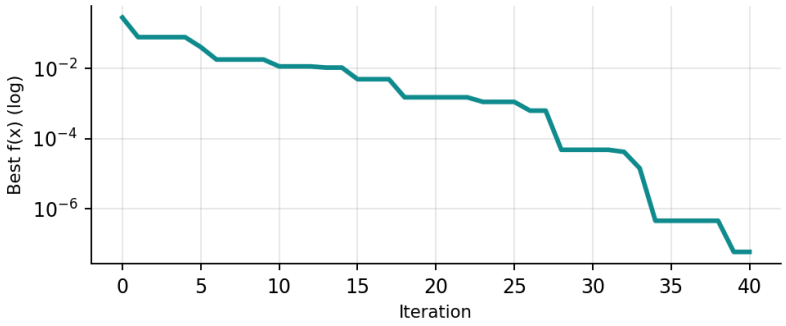


Fig 2.4 Convergence curve – best fitness falls rapidly

2.5 Neighbourhood topologies

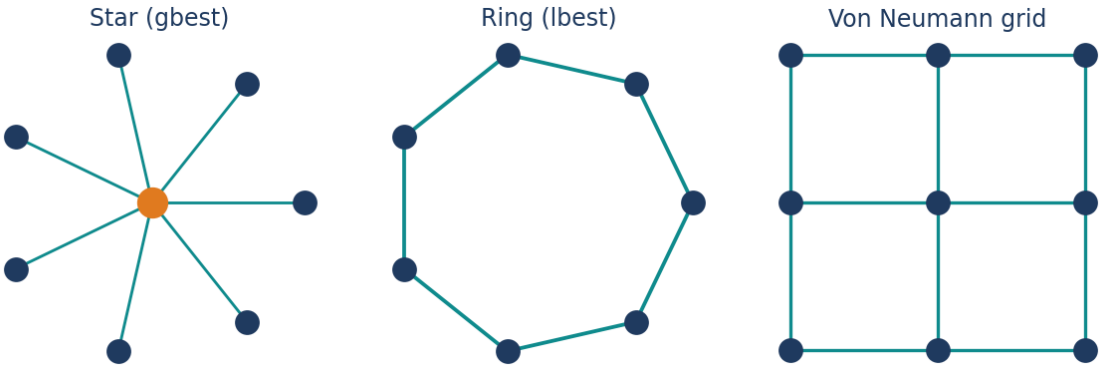


Fig 2.5 Who talks to whom: gbest (star), lbest (ring), grid

- **gbest (star):** every particle follows the global best → fast but may get trapped (premature convergence).
- **lbest (ring):** each particle follows only its neighbours → slower but explores better, avoids local optima.

2.5b Effect of inertia weight

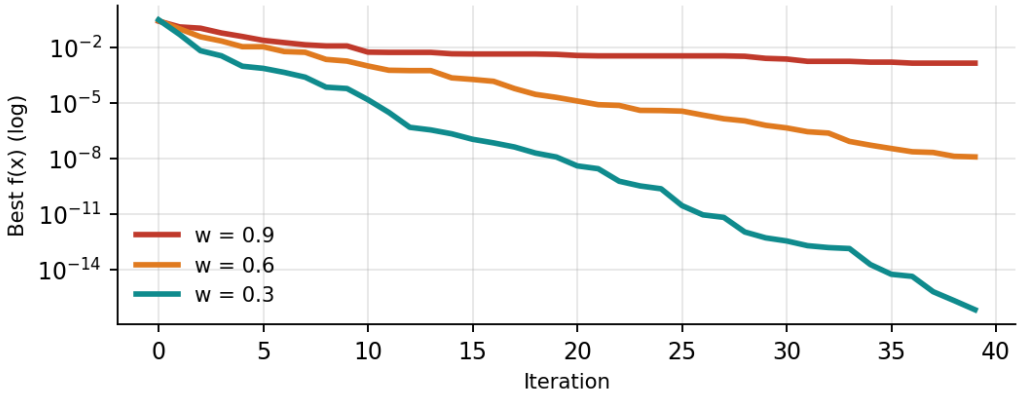


Fig 2.6 Average convergence for different inertia weights (simulation over 8 runs)

- **w large (0.9):** particles keep moving – good exploration, but slow and may overshoot.
- **w small (0.3):** fast settling – good exploitation, but risk of local optimum.
- Best practice: **start high and reduce linearly** (e.g. 0.9 → 0.4).

2.5c Important PSO variants

Variant	Idea
Constriction-factor PSO	A factor χ multiplies the whole velocity to guarantee convergence
Binary PSO	Velocity becomes a probability (sigmoid) that a bit is 1 – for discrete problems
Adaptive / dynamic PSO	w, c_1, c_2 change automatically during the run
Hybrid PSO-GA / PSO-DE	Adds mutation or crossover to keep diversity

2.5d Using PSO to tune a fuzzy controller

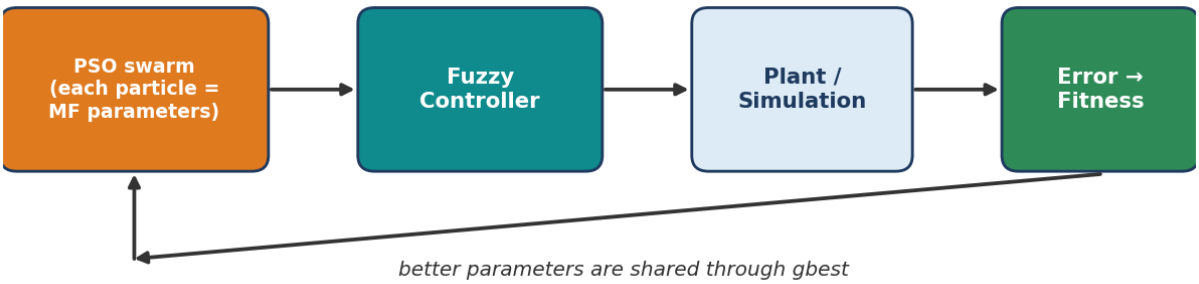


Fig 2.7 PSO searches the best membership-function parameters by minimising control error

2.6 PSO vs Genetic Algorithm

Point	PSO	GA
Inspiration	Bird flocking / social behaviour	Natural evolution
Operators	Velocity & position update	Selection, crossover, mutation
Memory	Particles remember pbest and gbest	Only the current population
Representation	Real numbers (mostly)	Binary / real / permutation
Parameters	Few (w, c_1, c_2)	More (pc, pm, selection type)
Speed	Usually faster convergence	Slower but more diverse

2.6b Practical issues in PSO

Issue	Remedy
Particles leave the search range	Clamp position to the bounds or reflect the particle back
Velocity explosion	Limit with v_{max} or use constriction factor
Premature convergence	Use lbest topology, decreasing w, or add mutation
When to stop?	Max iterations, fitness below a threshold, or no improvement for N iterations
Choosing swarm size	20–50 is enough for most problems; larger for high dimensions

2.7 Advantages, limitations, applications

- **Advantages:** simple, few parameters, easy to code, fast, no gradient needed.
- **Limitations:** can converge prematurely to local optimum; weak for discrete problems unless modified (binary PSO); parameter sensitivity.
- **Applications:** neural-network training, fuzzy-controller tuning, power-system dispatch, feature selection, image segmentation, robot path planning, antenna design.

Memory trick

PSO = Position + Speed + Own best + Group best. "I trust my past (c_1) and my friends (c_2), but I also keep some momentum (w)."

3 Solved Numerical Problems

Problem 1 – One full PSO iteration (3 particles)

Minimise $f(x) = x^2$. Start: x = (4, −3, 2), v = (1, −1, 0.5), pbest = x, so initial **gbest = 2** (f = 4). Take w = 0.5, $c_1 = c_2 = 1.0$, $r_1 = r_2 = 0.5$. Formula: $v' = w \cdot v + c_1 r_1(pbest - x) + c_2 r_2(gbest - x)$.

Particle	x	v	pbest	new v	new x	f(new x)	new pbest
P1	4.00	1.00	4.00	-0.50	3.50	12.25	3.50
P2	-3.00	-1.00	-3.00	2.00	-1.00	1.00	-1.00
P3	2.00	0.50	2.00	0.25	2.25	5.06	2.00

Example, P2: $v' = 0.5\times(-1) + 1\times0.5\times(-3-(-3)) + 1\times0.5\times(2-(-3)) = -0.5 + 0 + 2.5 =$ **2.00**; $x' = -3 + 2 =$ **-1.00**, f = 1.00 which is better than 9, so pbest becomes -1.00.

New gbest = -1.00 (f = 1.00). The swarm has already moved much closer to the minimum at x = 0.

Problem 2 – Fuzzy decision (max–min)

Goal μG = (0.7, 0.9, 0.4), constraint μC = (0.8, 0.5, 0.6) for alternatives (a, b, c).
μD = min(μG, μC) = (0.7, 0.5, 0.4) → **choose alternative a** (μD = 0.7).

Problem 3 – Weighted multi-criteria

Weights (0.6, 0.4). Alt X = (0.5, 0.9) → 0.3 + 0.36 = **0.66**. Alt Y = (0.8, 0.5) → 0.48 + 0.20 = **0.68**. **Y is selected**, though X is much better on criterion 2.

Common mistakes to avoid

• Using max instead of min for the Bellman–Zadeh decision. • Forgetting that r_1, r_2 are random each step. • Updating position before velocity. • Not updating pbest/gbest after the move.

4 One-page Revision & Important Questions

Formula sheet

• Fuzzy decision: $\mu_D = \min(\mu_G, \mu_C)$; best = argmax μ_D. • MCDM score = $\Sigma\, w_j\mu_{ij}$.
• PSO: $v \leftarrow w\cdot v + c_1r_1(pbest - x) + c_2r_2(gbest - x)$; $x \leftarrow x + v$.
• Large w = exploration; small w = exploitation. • gbest = fast; lbest = diverse.

Quick self-check

Q1. Which operator gives the Bellman–Zadeh decision? → **min (intersection)**
Q2. What does c_2 represent in PSO? → **Social learning (pull towards gbest)**
Q3. Who proposed PSO? → **Kennedy & Eberhart, 1995**
Q4. Name two problems of PSO. → **Premature convergence, poor on discrete problems**

Likely exam questions

- Explain fuzzy decision making. Describe the Bellman–Zadeh model with an example.
- Explain multi-person and multi-criteria fuzzy decision making.
- Explain PSO with algorithm / flowchart and the velocity update equation.

- Solve: compute new velocity and position of a particle (numerical).
- Compare PSO with GA; list advantages, limitations and applications of PSO.

DIGITAL NOTES • SOFT COMPUTING

UNIT – IV

Genetic Algorithms

What is inside this unit

1	Basic concepts and terminology, encoding
2	Genetic algorithm cycle (flowchart, termination, replacement)
3	Fitness function and scaling
4	Basic operators – selection, crossover, mutation
5	Crossover and mutation properties, schema theorem
6	Worked numerical example
7	Applications of GA
8	Revision sheet & exam questions

After this unit you will be able to

- Explain GA terminology and encode a problem.
- Describe the GA cycle and write its algorithm.
- Design a fitness function and choose a selection method.
- Apply crossover and mutation and discuss pc and pm.
- Solve a one-generation GA example by hand.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Genetic Algorithms – Basic Concepts

A **Genetic Algorithm (GA)** is a **search and optimisation technique based on natural selection and genetics**, introduced by **John Holland (1975)** and popularised by D. E. Goldberg. It evolves a population of candidate solutions over generations; **fitter solutions get more chances to reproduce**, so the population steadily improves.

- **Idea:** "survival of the fittest" + "inheritance of good genes" = search for the best solution.
- **Population-based:** works with many solutions at once, not just one point.
- **Uses probabilistic rules** (random selection, crossover, mutation) – not deterministic ones.
- **Needs only fitness values** – no derivatives or smoothness required.

1.1 Biological terms used in GA

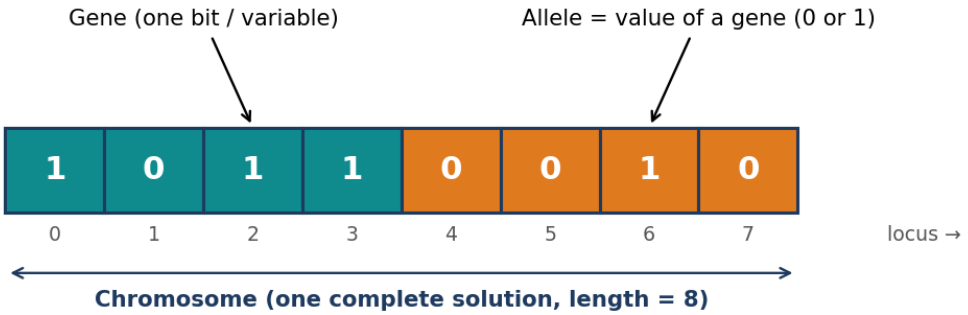


Fig 1.1 Anatomy of a chromosome

Term	Biology	In GA
Gene	Unit of heredity	One position (variable / bit) in the string
Allele	Value of a gene	0 or 1 (or a number / symbol)
Chromosome (individual)	DNA strand	One complete candidate solution (string)
Locus	Position of a gene	Index in the string
Population	Group of organisms	Set of chromosomes (size N)
Genotype	Genetic make-up	Encoded string, e.g. 01101
Phenotype	Observable body	Decoded solution, e.g. x = 13
Fitness	Survival ability	Quality of the solution (objective value)
Generation	One breeding cycle	One iteration of the GA

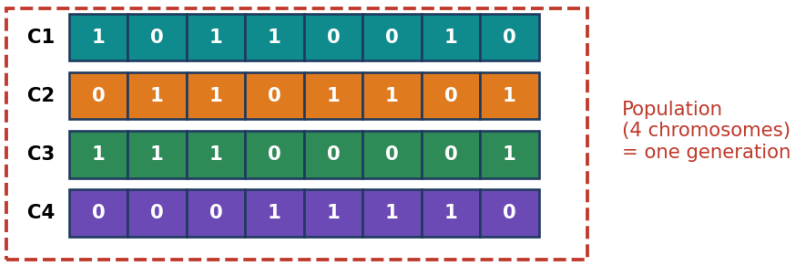


Fig 1.2 A population of four chromosomes

1.2 Encoding (representation) of solutions

The problem's variables must first be **encoded** as chromosomes. Good encoding makes crossover and mutation produce **valid** solutions.

Encoding	Example	Used for
Binary	1 0 1 1 0 → x = 22	Numerical optimisation, knapsack, feature selection
Value (real)	2.5 7.1 0.3	Continuous parameters, NN weights
Permutation	3 1 4 2 5	Travelling salesman, scheduling (order matters)
Tree	(+ (× x 2) y)	Genetic programming – evolving programs / formulas

Decoding a binary gene

For a variable in [a, b] using n bits: **$x = a + (b - a) \times$ (decimal value) / $(2^n - 1)$**. Example: n = 5, range [0, 31] → string 01101 = 13 → x = 13.

2 Genetic Algorithm Cycle

The GA repeats the same loop – **evaluate → select → crossover → mutate** – until a stopping rule is met.

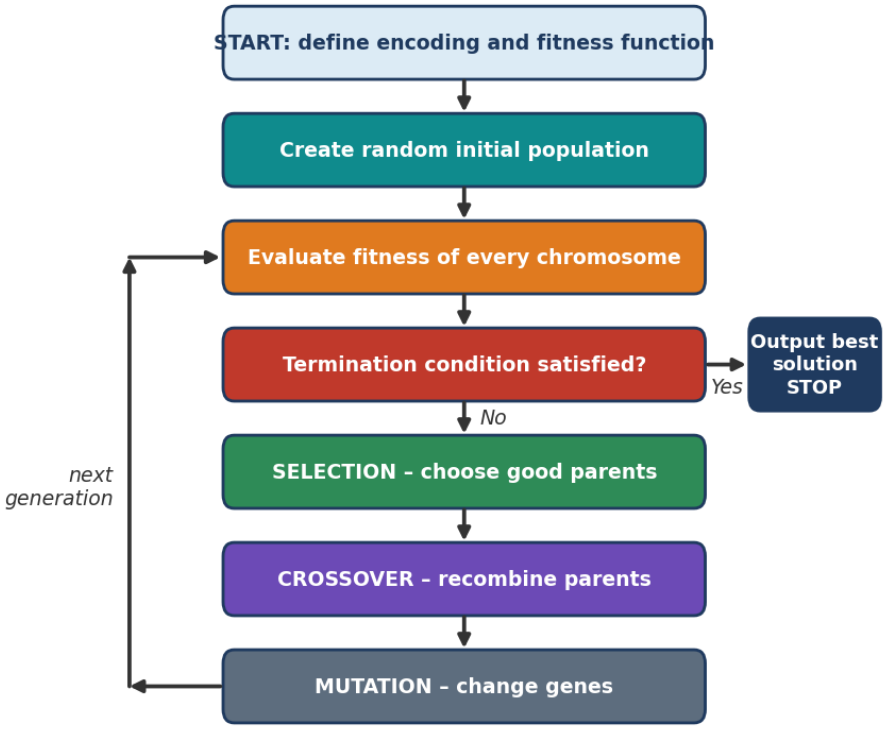


Fig 2.1 Flow chart of the genetic algorithm cycle

Genetic Algorithm
1. Choose encoding, population size N, pc, pm; generation t = 0
2. Initialise population P(0) randomly
3. Evaluate fitness of all individuals
4. **while** (stopping criterion not met):
   a) select parents (by fitness)
   b) crossover parents with probability pc → offspring
   c) mutate offspring genes with probability pm
   d) evaluate offspring; build P(t+1); t = t + 1
5. return the best chromosome found

2.1 Stopping (termination) criteria

- Fixed maximum number of generations reached.
- Fitness target achieved (good enough solution).
- No improvement in best fitness for many generations (convergence).
- Time or computing budget finished.

2.2 Replacement strategies

- **Generational:** the whole population is replaced by offspring.
- **Steady-state:** only a few worst members are replaced each time.
- **Elitism:** the best 1–2 chromosomes are copied unchanged to the next generation, so the best solution is never lost.

3 Fitness Function

The **fitness function** is the "judge" of the GA: it gives every chromosome a score showing **how good the solution is**. It is the only link between the GA and the real problem.

- **Maximisation:** fitness = objective value, e.g. $f(x) = x^2$.
- **Minimisation:** convert, e.g. fitness = 1 / (1 + cost) or C_{max} − cost.
- **Constraints:** subtract a **penalty** for violating a constraint: fitness = f(x) − penalty.

Qualities of a good fitness function

• **Correct** – higher score really means better solution. • **Fast to compute** – it is called thousands of times. • **Discriminating** – gives different scores to different quality solutions. • **Smooth enough** – small change in gene gives small change in score. • **Non-negative** – needed by roulette-wheel selection.

3.1 Fitness landscape

Imagine fitness as hills: the highest peak is the global optimum, smaller peaks are local optima. A GA starts with points scattered everywhere and gradually gathers them near the highest peaks.

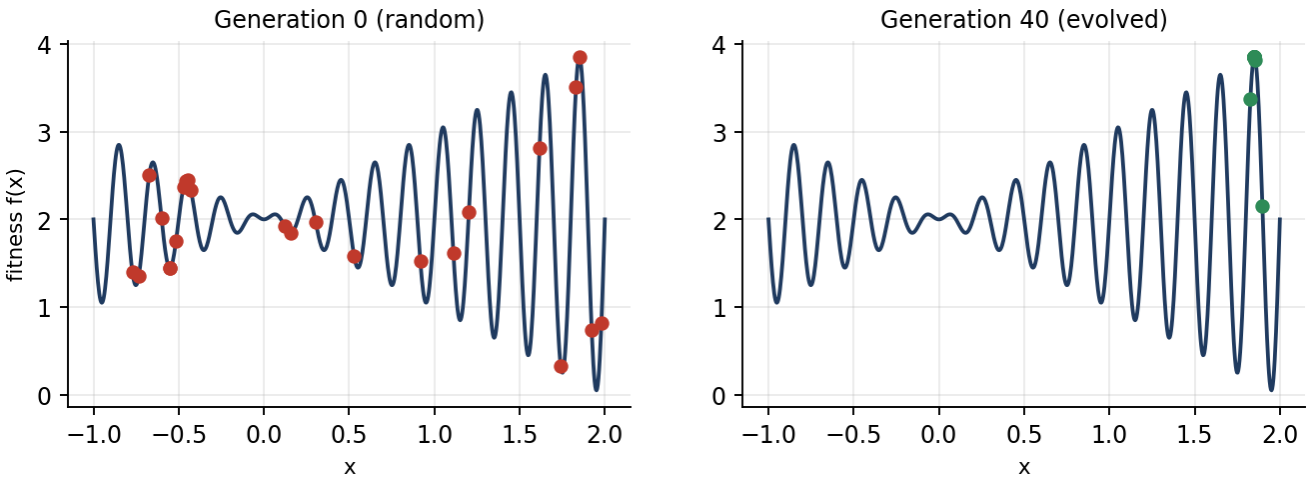


Fig 3.1 GA on f(x) = x·sin(10πx) + 2 : random start vs after 40 generations (simulation)

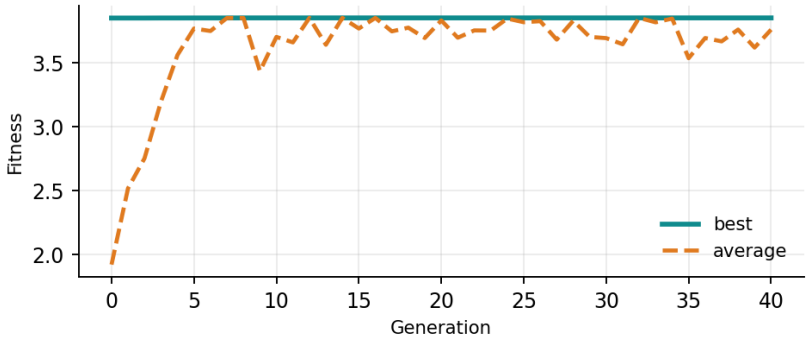


Fig 3.2 Convergence – best and average fitness rise and come together

3.2 Fitness scaling

- **Problem:** one super-fit chromosome early on dominates selection → premature convergence.
- **Linear scaling:** $f' = a \cdot f + b$ keeps average fitness fixed and limits the best to 1.5–2× the average.
- **Rank-based:** use the rank instead of raw fitness, so differences in magnitude do not matter.

4 Basic Operators of Genetic Algorithms

Three operators drive the GA: **Selection** (exploit good solutions), **Crossover** (combine them) and **Mutation** (explore new ones).

4.1 Selection (reproduction)

Chooses parents so that **better chromosomes are picked more often**, yet weaker ones keep a small chance (diversity).

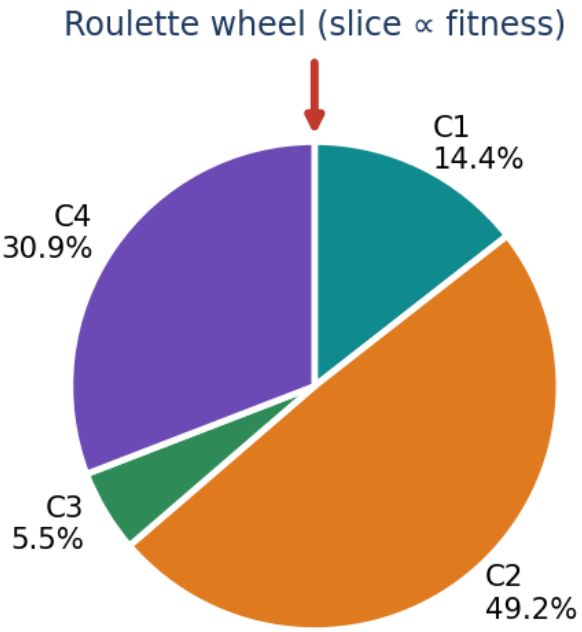


Fig 4.1 Roulette wheel selection for the example population (fitness 169, 576, 64, 361)

Method	How it works	Remark
Roulette wheel	Slice size ∝ fitness; spin N times. $p_i = f_i / \Sigma f$	Simple; fails if one fitness is huge
Tournament	Pick k random individuals; the best wins	Popular, no scaling needed, k = 2–5 sets pressure
Rank	Sort by fitness, probability by rank	Avoids domination
Stochastic universal sampling	One spin, N equally spaced pointers	Lower variance than roulette
Elitism	Copy best 1–2 directly	Never loses the best

4.2 Crossover (recombination)

Takes **two parents** and exchanges parts of their strings to create **two children**. It is the main search operator and is applied with probability **pc**.



Fig 4.2 Single-point crossover

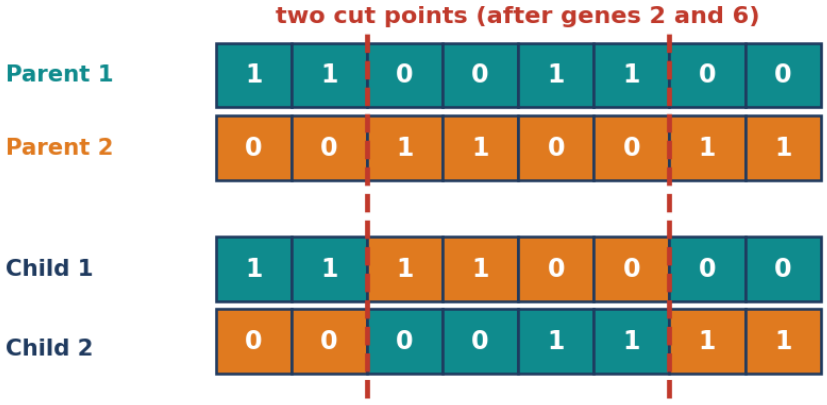


Fig 4.3 Two-point crossover

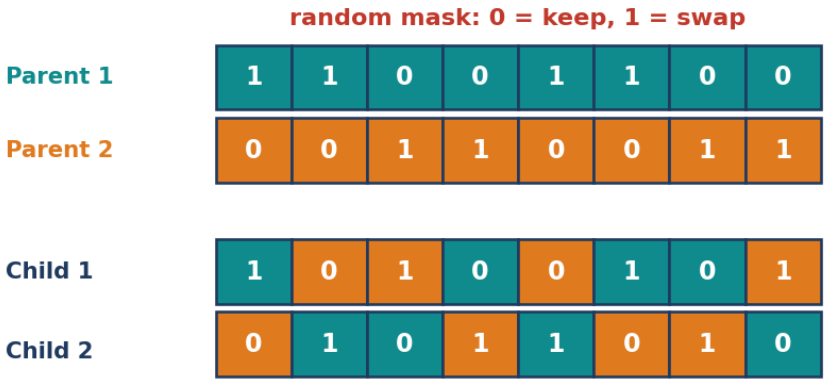


Fig 4.4 Uniform crossover (a random mask decides the source of every gene)

Type	Remark
Single-point	Simplest; one cut point; may break long building blocks
Two-point / k-point	Swaps middle segment; less positional bias
Uniform	Each gene from either parent (p = 0.5); high mixing
Arithmetic (real genes)	child = α·p1 + (1 − α)·p2
PMX / Order (OX)	For permutations – keep every city exactly once (TSP)

4.3 Mutation

Randomly changes genes with a **small probability pm**. It restores lost genes and adds **diversity**, so the GA can escape local optima.

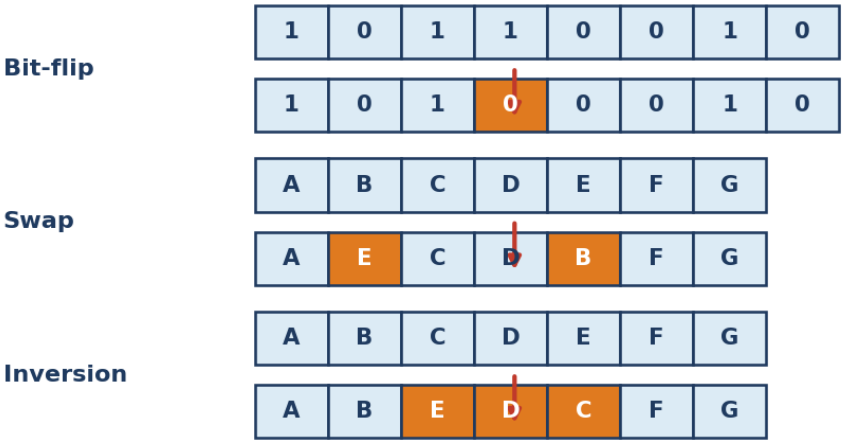


Fig 4.5 Mutation types: bit-flip (binary), swap and inversion (permutation)

- **Bit-flip:** 0 ↔ 1 for each gene with prob. pm. **Random reset / Gaussian:** for real genes, add small random noise.
- **Swap / Inversion:** for permutation chromosomes (keeps the tour valid).

5 Crossover and Mutation Properties

Property	Crossover	Mutation
Role	**Exploitation** – combines good parts of parents	**Exploration** – introduces new genetic material
Parents needed	Two (or more)	One
Typical rate	pc = 0.6 – 0.9 (high)	pm = 0.001 – 0.05, about 1 / chromosome length (low)
Effect on population	Moves population towards good regions fast	Prevents premature convergence and loss of genes
If too high	Good parents are disrupted	GA becomes a random search
If too low	Slow improvement	Stuck in local optimum
Alone	Cannot create a gene not present in parents	Too slow to build good solutions by itself

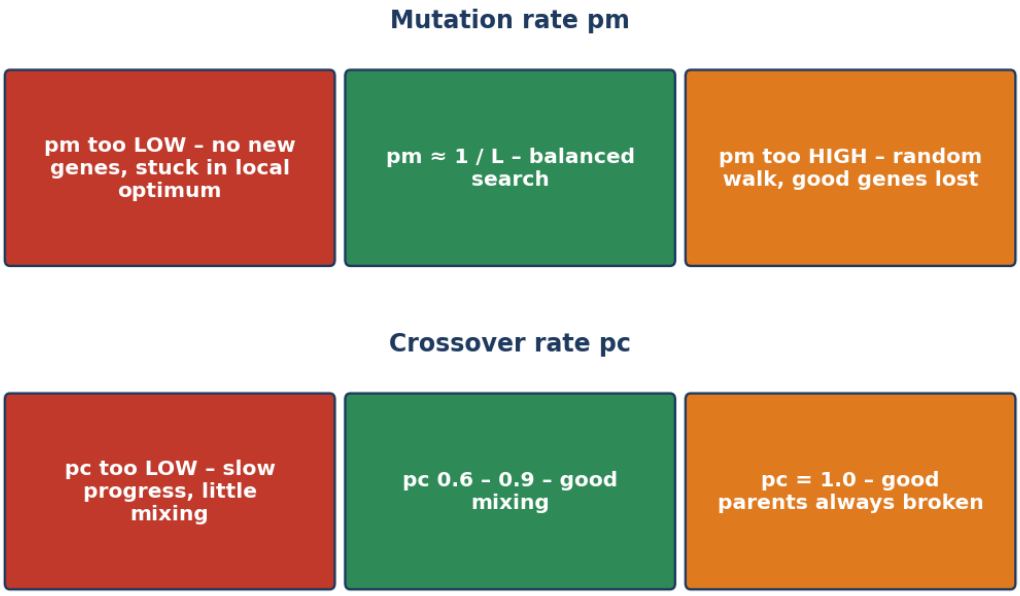


Fig 5.1 What happens when the rates are wrong

5.1 Why does a GA work? – Schema theorem

- A **schema** is a pattern with wildcards, e.g. **1 * * 0 *** (* = don't care). It stands for a whole family of strings.
- **Order** = number of fixed genes; **defining length** = distance between first and last fixed gene.
- **Schema theorem (Holland):** short, low-order, above-average schemata receive exponentially more copies in successive generations.
- These are the **building blocks**; crossover combines them into better and bigger solutions (**building block hypothesis**).

Memory trick

Crossover = "mix and match", Mutation = "small surprise". Together: crossover exploits, mutation explores.

6 Worked Example – Maximise f(x) = x²

Find x in [0, 31] that maximises f(x) = x². Use **5-bit binary** strings, population N = 4, roulette-wheel selection, single-point crossover.

Step 1 – Initial population and fitness

Chrom.	String	x	f(x) = x²	Probability f/Σf	Expected copies
C1	01101	13	169	0.144	0.58
C2	11000	24	576	0.492	1.97
C3	01000	8	64	0.055	0.22
C4	10011	19	361	0.309	1.23

Σf = 1170, average = 292.5, best = 576. **C2** (fittest) is expected to get ~2 copies, **C3** (weakest) ~0.

Step 2 – Selection (mating pool)

After roulette-wheel spins the mating pool is: **C1, C2, C2, C4** (C3 is dropped).

Step 3 – Crossover

Pair	Parents	Cut	Children	x values	f(x)
1	01101 , 11000	after gene 4	01100 , 11001	12 , 25	144 , 625
2	11000 , 10011	after gene 2	11011 , 10000	27 , 16	729 , 256

Step 4 – Mutation

With a small pm, suppose only the third gene of the last child flips: **10000 → 10100** (x = 20, f = 400).

Step 5 – New generation

	Strings	Σ f	Average f	Best f
Old	01101 11000 01000 10011	1170	292.5	576
New	01100 11001 11011 10100	1898	474.5	729

Result

In **one generation** the average fitness rose from 292.5 to 474.5 and the best from 576 to 729. Repeating the cycle drives the population towards **11111 (x = 31, f = 961)**, the true optimum.

7 Applications of Genetic Algorithms

Area	Problem solved with GA
Engineering design	Optimising aircraft wings, truss structures, antenna shapes, circuit layout (VLSI)
Scheduling & planning	Timetabling, job-shop scheduling, vehicle routing, travelling salesman
Machine learning	Feature selection, neural-network weights/architecture, tuning fuzzy rules (genetic-fuzzy)
Robotics & control	Path planning, controller parameter tuning, evolving robot behaviour
Finance & business	Portfolio optimisation, trading rules, supply-chain optimisation
Bioinformatics & medicine	Protein folding, gene-expression clustering, drug design, radiotherapy planning
Image & signal processing	Image segmentation, compression, filter design
Games & creative	Game AI, evolutionary art and music, level generation

7.1 Advantages and limitations

Advantages	Limitations
Needs no derivatives; works on discontinuous, noisy functions	No guarantee of the global optimum
Searches many points at once – good global search	Needs many fitness evaluations (slow if fitness is costly)
Handles many variables, constraints and multiple objectives	Choosing encoding and parameters needs experience
Easily combined with fuzzy, neural and other methods	Premature convergence if diversity is lost

7.2 GA vs traditional optimisation

Point	Traditional (calculus / gradient)	Genetic algorithm
Search	From a single point	From a population of points
Information used	Derivatives needed	Only fitness values
Rules	Deterministic	Probabilistic
Local optima	Easily trapped	Better chance to escape
Search space	Smooth, continuous	Any – discrete, noisy, discontinuous

8 One-page Revision & Important Questions

Formula and fact sheet

• Selection probability $p_i = f_i / \Sigma f$. Expected copies = $p_i \times N$. • Decode: $x = a + (b - a) \cdot D / (2^n - 1)$.
• GA cycle: Encode → Initial population → Fitness → Selection → Crossover → Mutation → New generation → Stop?
• pc high (0.6–0.9), pm low (≈ 1/L). • Elitism keeps the best. • Schema theorem: short, low-order, fit schemata multiply.
• Crossover = exploitation; mutation = exploration.

Quick self-check

Q1. Who introduced GA? → **John Holland, 1975**
Q2. Which operator brings new genes? → **Mutation**
Q3. What does a fitness function do? → **Scores quality of each chromosome**
Q4. Best encoding for TSP? → **Permutation**

Likely exam questions

- Explain the basic concepts and terminology of genetic algorithms.
- Draw and explain the GA cycle / flowchart.
- What is a fitness function? How is it designed? Explain fitness scaling.
- Explain selection methods (roulette, tournament, rank) with examples.
- Explain crossover and mutation types with diagrams; discuss their properties.
- Solve a numerical: one generation of GA for $f(x) = x^2$.
- List applications, advantages and limitations of GA.

DIGITAL NOTES • SOFT COMPUTING

UNIT – V

Rough Sets & Integration of Soft Computing

What is inside this unit

1	Rough sets – indiscernibility, lower/upper approximation, dependency
2	Reducts, core and the discernibility matrix
3	Rule induction – certain and possible rules, quality measures
4	Integration of soft computing techniques (neuro-fuzzy, genetic-fuzzy, neuro-genetic, rough hybrids)
5	Solved problem & revision sheet

After this unit you will be able to

- Compute lower and upper approximations and accuracy of a rough set.
- Build a discernibility matrix and find reducts and core.
- Generate IF–THEN rules from a decision table.
- Explain why and how soft-computing techniques are integrated.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Rough Sets

Rough set theory was proposed by **Zdzisław Pawlak (1982)** to handle **vague and incomplete knowledge** using **only the data itself**. Idea: objects that look the same (same attribute values) cannot be told apart – so a concept can often only be described by an **approximate (rough)** boundary.

Fuzzy sets	Rough sets
Vagueness described by **membership degree μ**	Vagueness described by **lower and upper approximations**
Needs a membership function (from expert)	**No extra information** needed – uses the data only
Handles gradual concepts (“tall”)	Handles indiscernibility and incomplete data
Output: degrees of truth	Output: reductions and decision rules

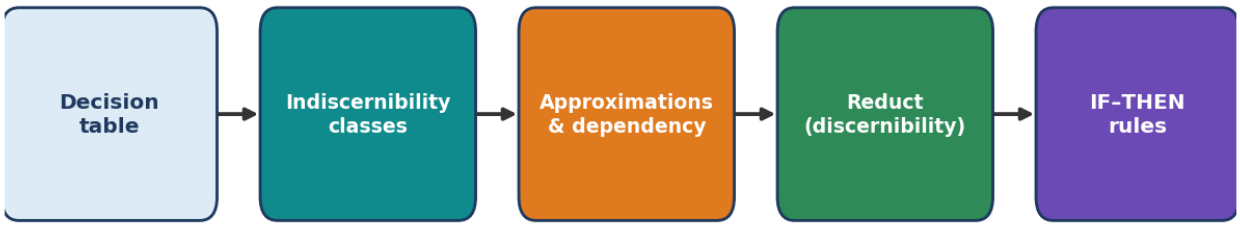


Fig 1.1 Overall rough-set analysis pipeline

1.1 Information system and decision table

Data is kept as a table. Rows = objects (U), columns = **condition attributes C** and one **decision attribute D**. Example – patients and flu:

Object	Headache (H)	Muscle-pain (M)	Temp (T)	Flu (D)
1	yes	yes	normal	**no**
2	yes	yes	high	**yes**
3	yes	yes	very high	**yes**
4	no	yes	normal	**no**
5	no	no	high	**no**
6	no	yes	very high	**no**

1.2 Indiscernibility relation

Two objects are **indiscernible** w.r.t. a set of attributes B if they have the **same values for all attributes in B**. This is an **equivalence relation**; it splits U into **equivalence classes** (granules) written $\mathbf{[x]_B}$.

Example

Take B = {Muscle-pain, Temp}. Objects 1 and 4 both have (yes, normal) → indiscernible. Objects 3 and 6 both have (yes, very high).
U / IND(B) = { {1,4}, {2}, {3,6}, {5} }

1.3 Lower and upper approximation

Let X ⊆ U be the concept we want to describe, e.g. X = patients with Flu = yes. Using the classes of B:

- **Lower approximation $\underline{B}X$** = union of classes **completely inside** X → objects that **certainly** belong to X.
- **Upper approximation $\overline{B}X$** = union of classes that **touch** X (overlap) → objects that **possibly** belong to X.
- **Boundary region $BN_B(X)$** = $\overline{B}X - \underline{B}X$ → cannot be decided. **X is rough if boundary ≠ ∅**, crisp (exact) if boundary = ∅.

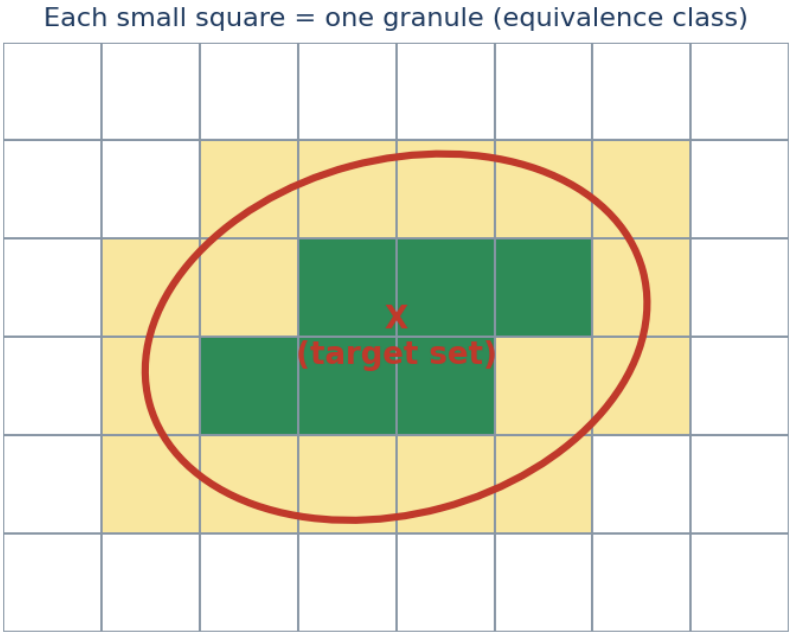


Fig 1.2 Granules of knowledge, the target set X and its approximations

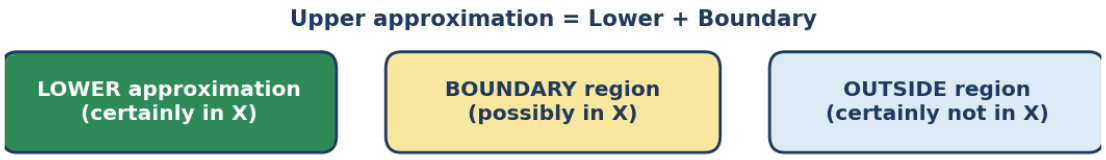


Fig 1.3 Colour key

Worked example – approximations of Flu = yes using B = {M, T}

X = {2,3} (patients 2 and 3 have flu). Classes: {1,4}, {2}, {3,6}, {5}
Lower $\underline{B}X$ = classes fully inside X = **{2}**
Upper $\overline{B}X$ = classes overlapping X = {2,3,6} = **{2,3,6}**
Boundary = {3,6} (objects 3 and 6 look identical on M and T, but 3 has flu and 6 does not!)
Outside = {1,4,5}
Accuracy of approximation α = |lower| / |upper| = 1/3 = 0.33 (1 = crisp, 0 = totally rough)

Formulas to remember

• $\alpha_B(X) = |\underline{B}X| / |\overline{B}X|$ • Roughness = 1 − α • Positive region $POS_B(D)$ = union of lower approximations of all decision classes
• Degree of dependency $\gamma_B(D) = |POS_B(D)| / |U|$ ($\gamma = 1 \rightarrow$ D fully depends on B)

1.4 Dependency of attributes

With B = {M, T}: positive region = {1,2,4,5} → **γ = 4/6 = 0.67** (objects 3 and 6 spoil the dependency). With all attributes C = {H, M, T}: every object is unique, so γ = 6/6 = **1** – the decision depends completely on the conditions.

1.5 Rough membership function

Rough sets can also give a **degree**: $\mu_X^B(x) = |[x]_B \cap X| / |[x]_B|$ – the fraction of the object's class that lies in X. Unlike a fuzzy μ, it is **computed from data**, not supplied by an expert.

Object	Class [x]	$\lvert[x]\cap X\rvert / \lvert[x]\rvert$	μ	Status
1	{1,4}	0/2	0.00	certainly not
2	{2}	1/1	1.00	certain in X
3	{3,6}	1/2	0.50	possible
4	{1,4}	0/2	0.00	certainly not
5	{5}	0/1	0.00	certainly not
6	{3,6}	1/2	0.50	possible

1.6 Extensions of rough sets

Extension	Idea
Variable precision rough set (VPRS)	Allows a small error β – a class counts as "inside" X if ≥ (1 − β) of it is in X (handles noise)
Fuzzy-rough sets	Equivalence classes and the target set are fuzzy – handles continuous data
Tolerance / similarity-based	Uses similarity instead of strict equality of values
Dominance-based (DRSA)	Handles ordered (preference) criteria such as low < medium < high

2 Reduces, Core, Discernibility Matrix

Not all attributes are needed. A **reduct** is a **minimal subset of condition attributes** that gives the **same classification power** (same γ) as the full set. The **core** is the set of attributes that are in **every** reduct – the indispensable ones: Core = ∩ Reducts.

2.1 Discernibility matrix

Build a symmetric matrix of objects. Each cell (i, j) lists **the condition attributes in which objects i and j differ**. For decision-relative reducts, fill only cells where objects have **different decisions** (others are "–").

	1	2	3	4	5	6
1						
2	T					
3	T	–				
4	–	H,T	H,T			
5	–	H,M	H,M,T	–		
6	–	H,T	H	–	–	

Fig 2.1 Decision-relative discernibility matrix for the flu table (lower triangle)

2.2 Discernibility function and reduct

- Write each non-empty cell as a **sum (OR)** of its attributes; combine all cells with **AND** (product).
- Simplify with Boolean laws (absorption: a · (a + b) = a). Each **prime implicant** = one **reduct**.

Worked example

Cells found: (H), (T), (H + M), (H + T), (H + M + T)
f(H, M, T) = T · (H + T) · (H + M) · (H + M + T) · H
Absorption: T · (H + T) = T, and H · (H + M) = H, etc. ⇒ **f = H · T**
Reduct = {H, T} = {Headache, Temp}, Core = {H, T}. **Muscle-pain (M) is redundant** and can be dropped without losing any classification power.

Exam tip – steps

1) Find all differing-attribute cells for object pairs with different decisions. 2) Write the CNF. 3) Simplify by absorption. 4) Prime implicants = reducts; single-attribute cells give the core.

2.3 QuickReduct – dependency-based search (worked)

Dependency of the decision on single attributes: γ(H) = 0.50, γ(M) = 0.17, γ(T) = 0.33. Take the best single attribute, then add the one that raises γ most:

Subset	γ	Comment
HM	0.50	
HT	1.00	γ = 1 → reduct found
MT	0.67	

The pair {H, T} reaches γ = 1 with only two attributes, which agrees with the discernibility-matrix result.

2.4 Other reduct methods

- **Dependency-based (QuickReduct):** start empty, add the attribute that increases γ most, stop when γ equals that of the full set.
- **Heuristic / evolutionary:** GA or PSO search subsets when the number of attributes is large (finding the shortest reduct is NP-hard).

3 Rule Induction

Rule induction converts the (reduced) decision table into **IF–THEN rules** that explain and predict the decision. Rough sets give rules that are **compact, readable and data-driven**.

3.1 Steps

- **1.** Compute a **reduct** and remove redundant attributes (here keep H, T).
- **2.** Write one rule per object from the reduced table: IF (attribute = value) AND … THEN decision.
- **3. Merge / simplify**: drop conditions that are not needed (value reducts) and remove duplicate rules.
- **4.** Measure rule quality: support, confidence, coverage.

Rule	IF … THEN …	Objects	Support	Confidence
R1	IF Headache = no THEN Flu = no	4, 5, 6	3	100 %
R2	IF Headache = yes AND Temp = normal THEN Flu = no	1	1	100 %
R3	IF Headache = yes AND Temp = high THEN Flu = yes	2	1	100 %
R4	IF Headache = yes AND Temp = very high THEN Flu = yes	3	1	100 %

Rules R3 and R4 may be merged: **IF Headache = yes AND Temp ∈ {high, very high} THEN Flu = yes** (support 2). Four short rules replace the whole six-row table, and the

attribute Muscle-pain is no longer used.

3.2 Certain and possible rules

Type	Made from	Confidence	Example (using B = {M, T})
Certain (exact) rule	Lower approximation	100 %	IF Muscle-pain = yes AND Temp = high THEN Flu = yes (object 2)
Possible (approximate) rule	Boundary / upper approximation	< 100 %	IF Muscle-pain = yes AND Temp = very high THEN Flu = yes (50 % – objects 3, 6)

3.3 Rule quality measures

Measure	Formula	Meaning
Support	|objects matching IF and THEN|	How many objects back the rule
Confidence (accuracy)	|IF ∧ THEN| / |IF|	Reliability of the rule
Coverage	|IF ∧ THEN| / |objects of that decision|	How much of the class is explained

Example R1: IF H = no (objects 4,5,6) THEN Flu = no. Support = 3, confidence = 3/3 = 100 %, coverage = 3/4 = 75 % (4 patients have no flu).

3.4 Applications of rough sets

- **Feature selection / dimensionality reduction** before neural networks or fuzzy systems.
- Medical diagnosis, risk analysis, bankruptcy and credit prediction.
- Data mining, text classification, web usage mining, image segmentation, fault diagnosis.
- Decision support where **explainable rules** are required.

3.5 Advantages and limitations

Advantages	Limitations
Needs no prior information (no μ, no probability)	Works on **discrete** data – continuous attributes must be discretised
Finds minimal attribute sets and readable rules	Sensitive to noise; strict indiscernibility
Simple, easy to understand mathematically	Reduct finding is NP-hard for large tables

4 Integration of Soft Computing Techniques

Each soft-computing tool has weaknesses. **Integration (hybridisation)** combines two or more tools so that **one's strength covers another's weakness**. This is Zadeh's vision: SC is a **partnership**, not a competition.

Tool	Strength	Weakness
Fuzzy logic	Human-like knowledge, handles vagueness	Cannot learn; rules designed by hand
Neural network	Learns from data, generalises	Black box; needs lots of data
Genetic algorithm / PSO	Global optimisation	Slow; no knowledge representation
Rough sets	Reducts and rules from raw data	Needs discrete data

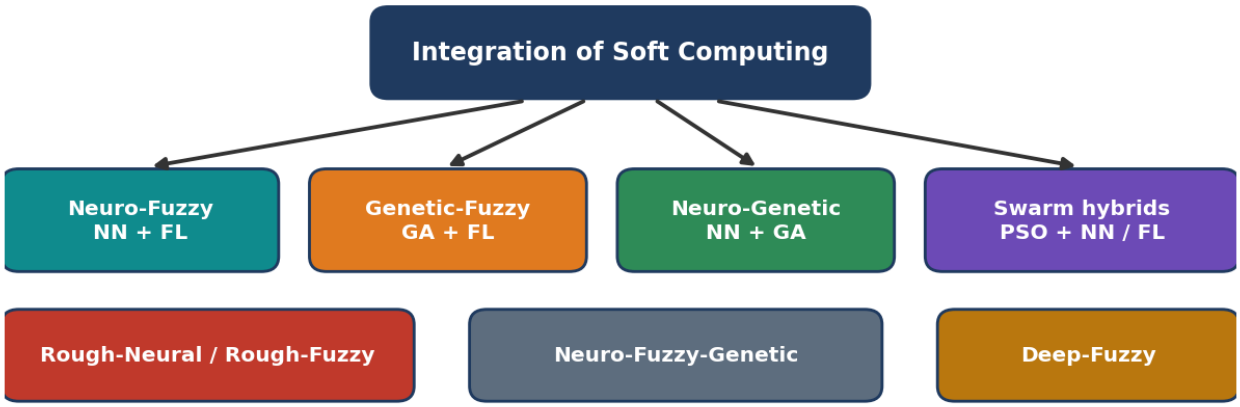


Fig 4.1 Families of hybrid soft-computing systems

4.1 Neuro-fuzzy systems (NN + FL)

- A neural network **learns** the membership functions and rules of a fuzzy system → **learning + interpretability**.
- Best known: **ANFIS** (Adaptive Neuro-Fuzzy Inference System, Jang 1993) – a Sugeno fuzzy system implemented as a 5-layer network.

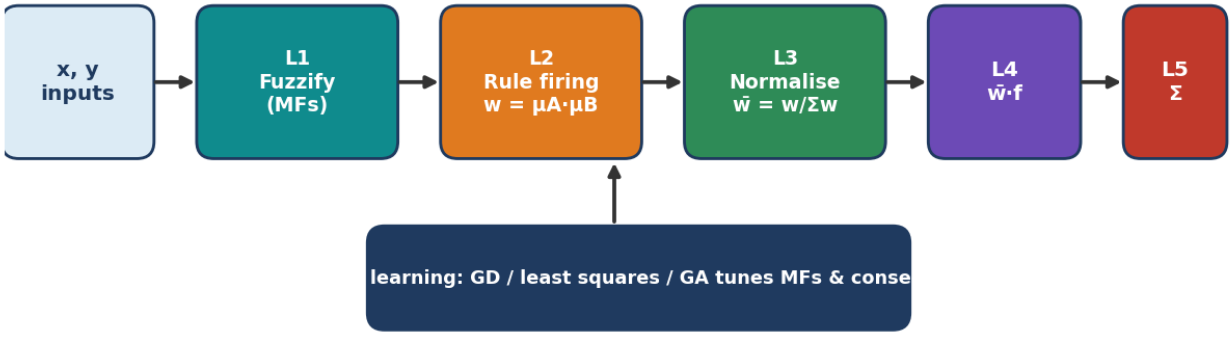


Fig 4.2 ANFIS architecture (5 layers)

Layer	Function
1 – Fuzzification	Node outputs μ(x) of input to a fuzzy set (premise parameters adapt)
2 – Rule layer	Firing strength w = AND of memberships (product)
3 – Normalisation	$\bar{w} = w / \Sigma w$
4 – Consequent	$\bar{w} \cdot f$, where $f = p \cdot x + q \cdot y + r$ (consequent parameters adapt)
5 – Output	$\Sigma \bar{w} \cdot f \rightarrow$ crisp output

Worked example – one ANFIS (Sugeno) forward pass

Rules: R1: IF x is A_1 THEN $f_1 = 2x + 1$ R2: IF x is A_2 THEN $f_2 = x + 4$. Input x = 3, with $\mu_{A_1}(3) = 0.6$, $\mu_{A_2}(3) = 0.2$.
Layer 2: $w_1 = 0.6$, $w_2 = 0.2$. Layer 3: $\bar{w}_1 = 0.6/0.8 = 0.75$, $\bar{w}_2 = 0.25$.
Layer 4: $f_1 = 2 \cdot 3 + 1 = 7$, $f_2 = 3 + 4 = 7 \rightarrow \bar{w}_1 f_1 = 5.25$, $\bar{w}_2 f_2 = 1.75$.
Layer 5 output = 5.25 + 1.75 = 7.0. During training, the parameters of A_1, A_2 and the coefficients of f_1, f_2 are adjusted to reduce error.

4.2 Genetic-fuzzy and swarm-fuzzy systems (GA/PSO + FL)

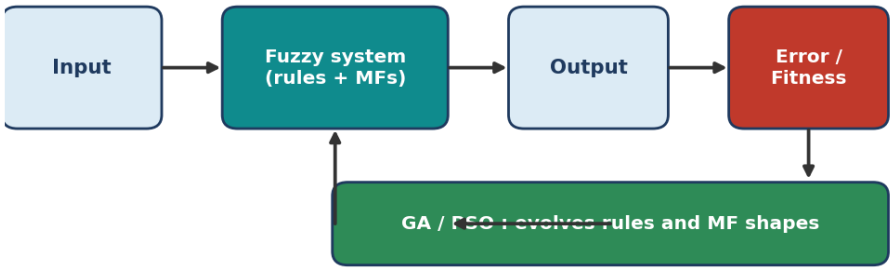


Fig 4.3 GA or PSO automatically designs the fuzzy controller

- The chromosome / particle encodes **MF shapes, number of rules** or rule consequents.
- Fitness = control error or classification accuracy; the best fuzzy system evolves automatically.
- Removes the long manual trial-and-error tuning of fuzzy controllers.

4.3 Neuro-genetic systems (NN + GA)

- GA **trains weights** (avoids local minima of back-propagation) or **chooses the network architecture** (layers, neurons, learning rate).
- Modern form: neuro-evolution and Neural Architecture Search.

4.4 Rough-neural and rough-fuzzy systems

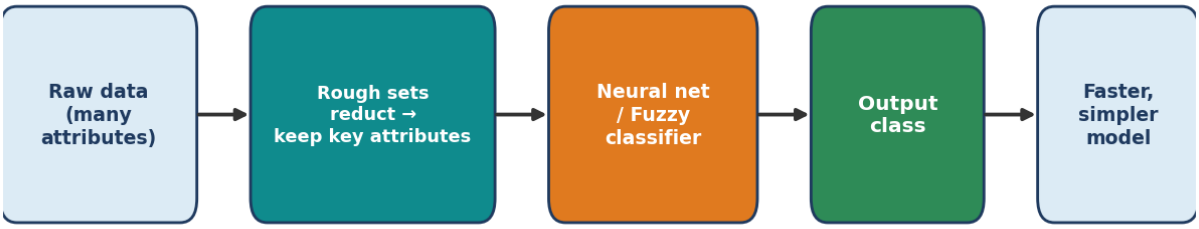


Fig 4.4 Rough sets reduce the attributes before a neural / fuzzy model is built

- **Rough sets** remove irrelevant attributes → smaller, faster, more accurate neural/fuzzy model.
- **Rough-fuzzy:** fuzzy granules + rough approximations handle both vagueness and indiscernibility.
- Rules from rough sets can be used to **initialise** a fuzzy or neural system.

4.5 Ways of combining (coupling)

Type	How	Example
Sequential (loose)	Output of one tool is input of the next	Rough-set reduct → neural classifier
Auxiliary	One tool helps tune another	GA tunes fuzzy MFs
Embedded (tight)	Tools share one structure	ANFIS (fuzzy inside a network)
Fusion / parallel	Tools run side by side and outputs combined	Ensemble of NN + fuzzy classifiers

Hybrid	Typical application
Neuro-fuzzy (ANFIS)	Time-series prediction, adaptive control, medical diagnosis
Genetic-fuzzy	Automatic controller design, rule-base learning, scheduling
Neuro-genetic	Weight optimisation, architecture search, stock prediction
Rough-neural / rough-fuzzy	Feature selection, data mining, gene-expression analysis
Swarm-neural	Training networks with PSO, power-system forecasting

Remember

Fuzzy + NN → learn and explain. GA/PSO + Fuzzy or NN → tune and design. Rough sets + others → clean data and extract rules.

5 Solved Problem & Revision

Problem – rough set analysis

Universe U = {a, b, c, d, e}. Using attribute set B the classes are {a, b}, {c}, {d, e}. Let X = {a, c, d}.
Lower = classes fully inside X = {c}. **Upper** = classes touching X = {a, b} ∪ {c} ∪ {d, e} = {a, b, c, d, e}.
Boundary = {a, b, d, e}. **Accuracy** = 1/5 = 0.2. X is a **rough set** (boundary not empty).

5.1 Full worked problem – buying a used car

Car	Price (P)	Mileage (M)	Colour (C)	Buy?
1	low	low	red	**yes**
2	low	high	red	**no**
3	high	low	blue	**yes**
4	high	high	blue	**no**
5	low	low	blue	**yes**
6	high	high	red	**no**
7	low	high	blue	**no**

Step 1 – approximations of X = Buy-yes = {1,3,5} using a single attribute

Attribute used	Classes	Lower	Upper	Accuracy
P = Price	{1,2,5,7}, {3,4,6}	∅	{1,2,3,4,5,6,7}	0/7 = 0.00
M = Mileage	{1,3,5}, {2,4,6,7}	{1,3,5}	{1,3,5}	3/3 = 1.00
C = Colour	{1,2,6}, {3,4,5,7}	∅	{1,2,3,4,5,6,7}	0/7 = 0.00

Mileage gives lower = upper = X, so X is **crisp** (accuracy 1): Mileage alone explains the decision. Price and Colour leave a large boundary.

Step 2 – discernibility: cells found: (M), (M+C), (P+M), (P+M+C). Single-attribute cell (M) → **core = {M}**; discernibility function f = M, so the **reduct = {M}**.

Step 3 – rules: R1: IF Mileage = low THEN Buy = yes (support 3, confidence 100 %). R2: IF Mileage = high THEN Buy = no (support 4, confidence 100 %).

Conclusion

Three attributes were reduced to one. Two rules explain all seven cars – a small, readable model found from data alone.

Problem – discernibility function

Suppose the discernibility cells are (a + b), (a + c) and (b + c). Then f = (a + b)(a + c)(b + c) = **ab + ac + bc**.
There are **three reducts**: {a, b}, {a, c}, {b, c}. No single attribute appears in every cell, so the **core is empty**.

Revision sheet

• Rough set (Pawlak 1982): approximate a concept by lower and upper approximation from data only.
• α = |lower| / |upper|; γ = |POS| / |U|. • Boundary empty → crisp set; else rough.
• Reduct = minimal attribute subset with same γ; core = common to all reducts. • Discernibility matrix → CNF → absorption → reducts.
• Rules: certain (lower), possible (boundary); quality = support, confidence, coverage.
• Hybrids: neuro-fuzzy (ANFIS), genetic-fuzzy, neuro-genetic, rough-neural, swarm-based.

Quick self-check

Q1. Who proposed rough sets? → **Z. Pawlak, 1982**
Q2. What is in the boundary region? → **Objects that cannot be classified with certainty**
Q3. What does the core contain? → **Attributes common to all reducts**
Q4. What does ANFIS combine? → **Fuzzy inference + neural learning**

Likely exam questions

- Explain rough sets; define lower, upper approximation and boundary with an example.
- What is a reduct and core? Find them using a discernibility matrix for the given table.
- Explain rule induction in rough sets; give certain and possible rules.
- Explain the integration of soft computing techniques; describe neuro-fuzzy (ANFIS) and genetic-fuzzy systems.
- Compare rough sets and fuzzy sets; list applications.